JavaGuide面试突击

（2025最新版）

JVM篇

公众号：JavaGuide

网站：javaguide.cn

前言

由于很多读者都有突击面试的需求，所以我在几年前就弄了 **JavaGuide 面试突击版本**（JavaGuide 内容精简版，只保留重点），并持续完善跟进。对于喜欢纸质阅读的朋友来说，也可以打印出来，整体阅读体验非常高！

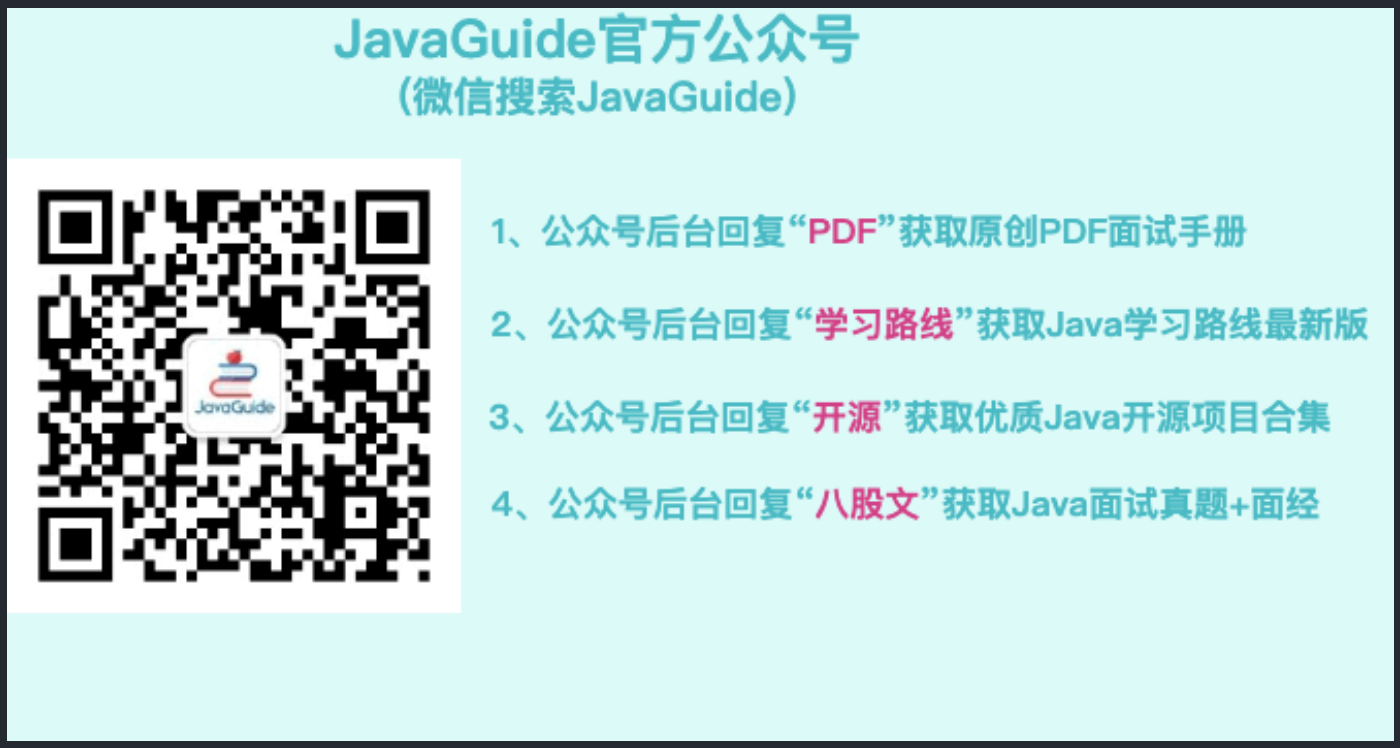
除了只保留最常问的面试题之外，我还进一步对重点中的重点进行了⭐标注。并且，有亮色（白天）和暗色（夜间）两个主题选择，需要打印出来的朋友记得选择亮色版本。

对于时间比较充裕的朋友，我个人还是更推荐 JavaGuide 网站系统学习，内容更全面，更深入。

JavaGuide 已经持续维护 6 年多了，累计提交了接近 **6000** commit ，共有 **570+** 多位贡献者共同参与维护和完善。用心做原创优质内容，如果觉得有帮助的话，欢迎点赞分享！传送门：GitHub | Gitee。

对于需要更进一步面试辅导服务的读者，欢迎加入 **JavaGuide 官方知识星球**(技术专栏/一对一提问/简历修改/求职指南/面试打卡)，绝对物超所值！

面试突击最新版本可以在我的公众号回复“**PDF**”获取（JavaGuide 官方知识星球会提前同步最新版，针对球友的一个小福利）。



这部分内容摘自 JavaGuide 下面几篇文章中的重点：

1. Java 内存区域详解

2. JVM 垃圾回收详解
3. 类文件结构详解
4. 类加载过程详解
5. 类加载器详解

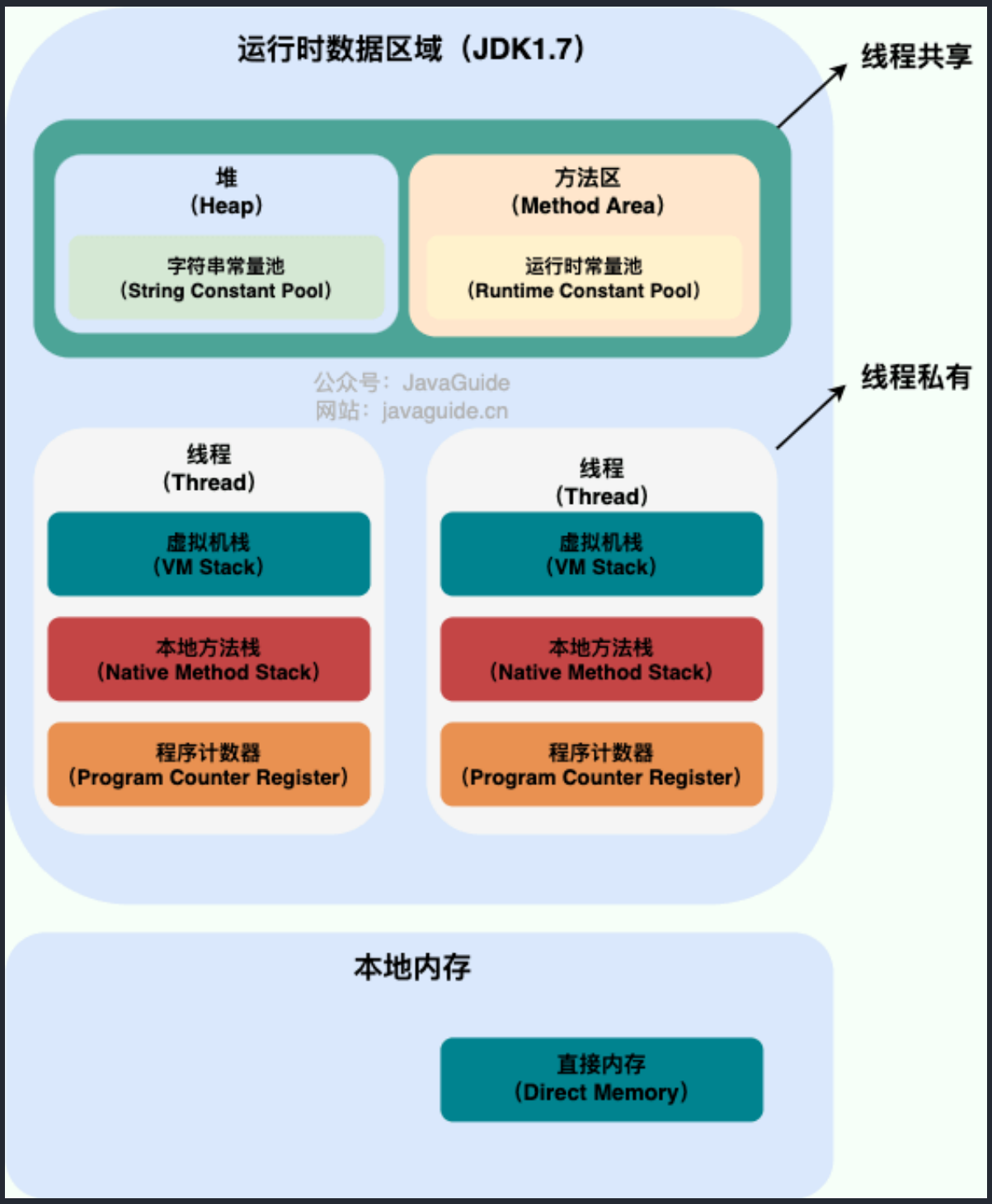
Java 内存区域

⭐Java 内存区域（运行时数据区）的组成

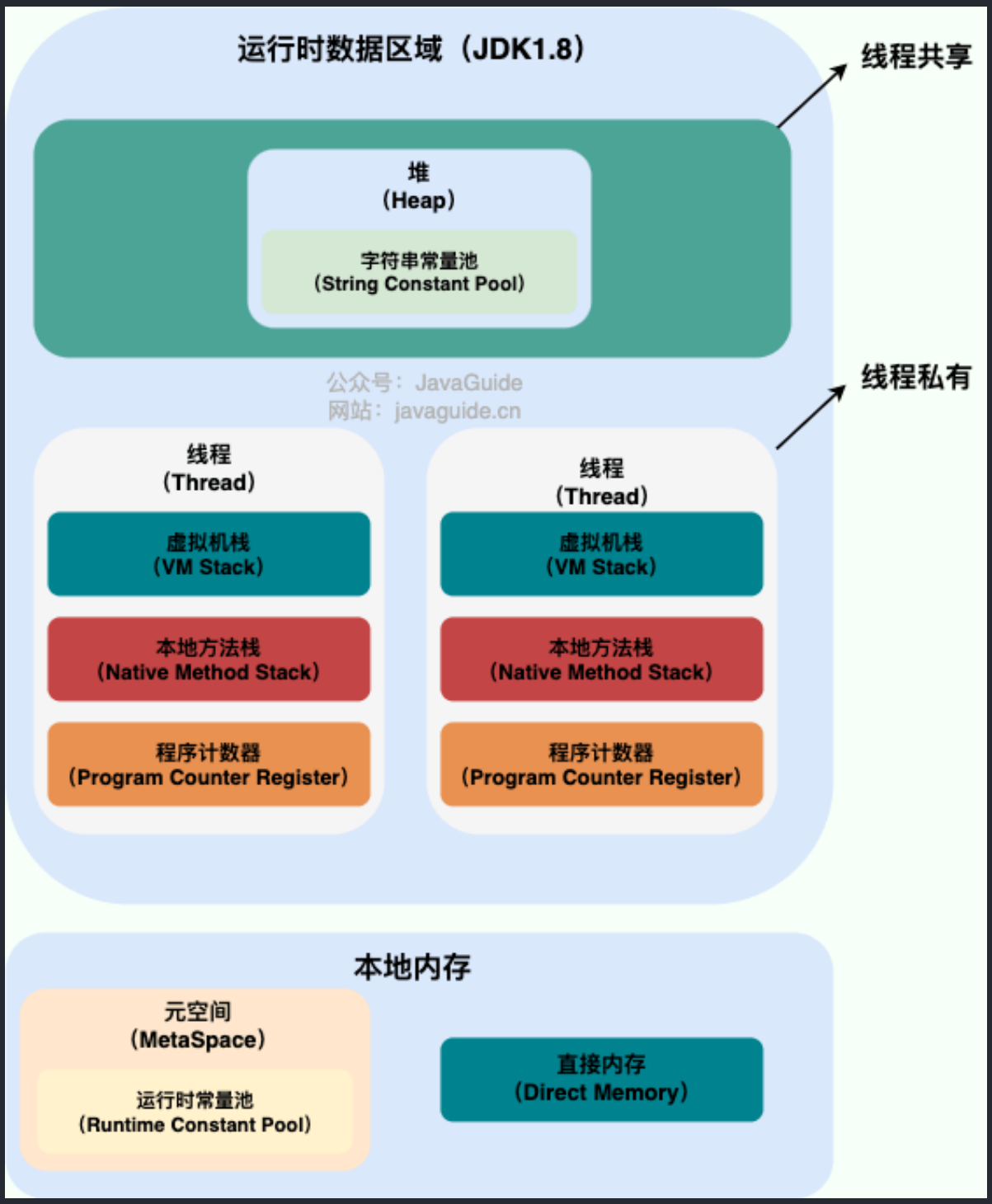
Java 虚拟机在执行 Java 程序的过程中会把它管理的内存划分成若干个不同的数据区域。

JDK 1.8 和之前的版本略有不同，我们这里以 JDK 1.7 和 JDK 1.8 这两个版本为例介绍。

JDK 1.7：


运行时数据区域（JDK1.7）
线程共享
堆
(Heap)
字符串常量池
(String Constant Pool)
方法区
(Method Area)
运行时常量池
(Runtime Constant Pool)
线程私有
公众号：JavaGuide
网站：javaguide.cn
线程
(Thread)
虚拟机栈
(VM Stack)
本地方法栈
(Native Method Stack)
程序计数器
(Program Counter Register)
线程
(Thread)
虚拟机栈
(VM Stack)
本地方法栈
(Native Method Stack)
程序计数器
(Program Counter Register)
本地内存
直接内存
(Direct Memory)

JDK 1.8：



线程私有的：

- 程序计数器
- 虚拟机栈
- 本地方法栈

线程共享的：

- 堆

- 方法区
- 直接内存 (非运行时数据区的一部分)

Java 虚拟机规范对于运行时数据区域的规定是相当宽松的。以堆为例：堆可以是连续空间，也可以不连续。堆的大小可以固定，也可以在运行时按需扩展 。虚拟机实现者可以使用任何垃圾回收算法管理堆，甚至完全不进行垃圾收集也是可以的。

⭐哪个区域不会出现 OutOfMemoryError?

程序计数器是唯一一个不会出现 `OutOfMemoryError` 的内存区域，它的生命周期随着线程的创建而创建，随着线程的结束而死亡。

程序计数器是一块较小的内存空间，可以看作是当前线程所执行的字节码的行号指示器。字节码解释器工作时通过改变这个计数器的值来选取下一条需要执行的字节码指令，分支、循环、跳转、异常处理、线程恢复等功能都需要依赖这个计数器来完成。

另外，为了线程切换后能恢复到正确的执行位置，每条线程都需要有一个独立的程序计数器，各线程之间计数器互不影响，独立存储，我们称这类内存区域为“线程私有”的内存。

从上面的介绍中我们知道了程序计数器主要有两个作用：

- 字节码解释器通过改变程序计数器来依次读取指令，从而实现代码的流程控制，如：顺序执行、选择、循环、异常处理。
- 在多线程的情况下，程序计数器用于记录当前线程执行的位置，从而当线程被切换回来的时候能够知道该线程上次运行到哪儿了。

程序运行中栈可能会出现什么错误?

- **`StackOverFlowError`**： 如果栈的内存大小不允许动态扩展，那么当线程请求栈的深度超过当前 Java 虚拟机栈的最大深度的时候，就抛出 `StackOverFlowError` 错误。
- **`OutOfMemoryError`**： 如果栈的内存大小可以动态扩展， 那么当虚拟机在动态扩展栈时无法申请到足够的内存空间，则抛出 `OutOfMemoryError` 异常。

在《Java虚拟机规范》中，对这个内存区域规定了两类异常状况：如果线程请求的栈深度大于虚拟机所允许的深度，将抛出StackOverflowError异常；如果Java虚拟机栈容量可以动态扩展[2]，当栈扩展时无法申请到足够的内存会抛出OutOfMemoryError异常。

[1] 栈帧是方法运行期很重要的基础数据结构，在本书的第8章中还会对帧进行详细讲解。
[2] HotSpot虚拟机的栈容量是不可以动态扩展的，以前的Classic虚拟机倒是可以。所以在HotSpot虚拟机上是不会由于虚拟机栈无法扩展而导致OutOfMemoryError异常——只要线程申请栈空间成功了就不会有OOM，但是如果申请时就失败，仍然是会出现OOM异常的，后面的实战中笔者也演示了这种情况。本书第2版时这里的描述是有误的，请阅读过第2版的读者特别注意。

堆内存的作用和组成

Java 虚拟机所管理的内存中最大的一块，Java 堆是所有线程共享的一块内存区域，在虚拟机启动时创建。**此内存区域的唯一目的就是存放对象实例，几乎所有的对象实例以及数组都在这里分配内存。**

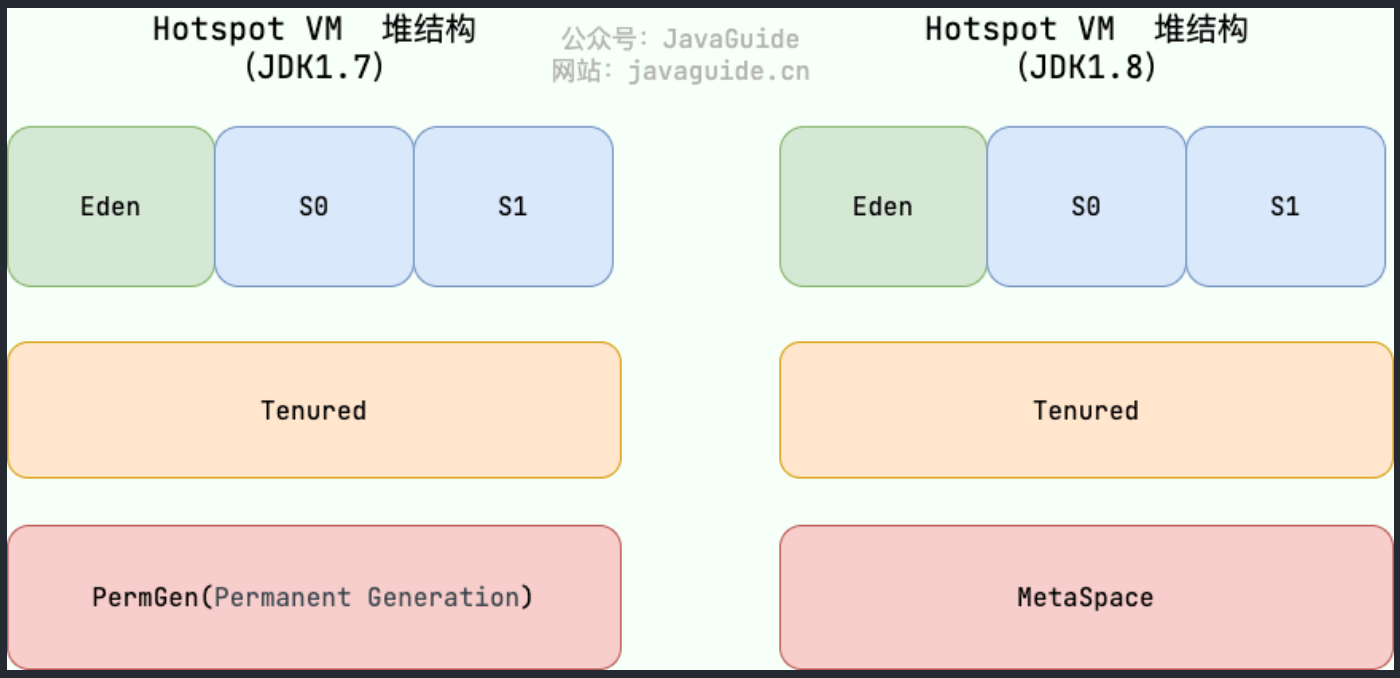
Java 世界中“几乎”所有的对象都在堆中分配，但是，随着 JIT 编译器的发展与逃逸分析技术逐渐成熟，栈上分配、标量替换优化技术将会导致一些微妙的变化，所有的对象都分配到堆上也渐渐变得不那么“绝对”了。从 JDK 1.7 开始已经默认开启逃逸分析，如果某些方法中的对象引用没有被返回或者未被外面使用（也就是未逃逸出去），那么对象可以直接在栈上分配内存。

Java 堆是垃圾收集器管理的主要区域，因此也被称作 **GC 堆（Garbage Collected Heap）**。从垃圾回收的角度，由于现在收集器基本都采用分代垃圾收集算法，所以 Java 堆还可以细分为：新生代和老年代；再细致一点有：Eden、Survivor、Old 等空间。进一步划分的目的是更好地回收内存，或者更快地分配内存。

在 JDK 7 版本及 JDK 7 版本之前，堆内存被通常分为下面三部分：

1. 新生代内存(Young Generation)
2. 老生代(Old Generation)
3. 永久代(Permanent Generation)

下图所示的 Eden 区、两个 Survivor 区 S0 和 S1 都属于新生代，中间一层属于老年代，最下面一层属于永久代。



JDK 8 版本之后 PermGen(永久代) 已被 Metaspace(元空间) 取代，元空间使用的是本地内存。（我会在方法区这部分内容详细介绍到）。

大部分情况，对象都会首先在 Eden 区域分配，在一次新生代垃圾回收后，如果对象还存活，则会进入 S0 或者 S1，并且对象的年龄还会加 1(Eden 区->Survivor 区后对象的初始年龄变为 1)，当它的年龄增加到一定程度（默认为 15 岁），就会被晋升到老年代中。对象晋升到老年代的年龄阈值，可以通过参数 `-XX:MaxTenuringThreshold` 来设置。不过，设置的值应该在 0-15，否则会爆出以下错误：

```
MaxTenuringThreshold of 20 is invalid; must be between 0 and 15
```

⭐程序运行中堆可能会出现什么错误?

堆这里最容易出现的就是 `OutOfMemoryError` 错误，并且出现这种错误之后的表现形式还会有几种，比如：

1. **`java.lang.OutOfMemoryError: GC Overhead Limit Exceeded`**：当 JVM 花太多时间执行垃圾回收并且只能回收很少的堆空间时，就会发生此错误。
2. **`java.lang.OutOfMemoryError: Java heap space`** :假如在创建新的对象时, 堆内存中的空间不足以存放新创建的对象, 就会引发此错误。(和配置的最大堆内存有关，且受制于物理内存大小。最大堆内存可通过 `-Xmx` 参数配置，若没有特别配置，将会使用默认值，详见：Default Java 8 max heap size)
3. ……

⭐为什么要将永久代 (PermGen) 替换为元空间 (MetaSpace) 呢?

下图来自《深入理解 Java 虚拟机》第 3 版 2.2.5

虚拟机时，但因为两者对方法区实现的差异而面临诸多困难。考虑到HotSpot未来的发展，在JDK 6的时候HotSpot开发团队就有放弃永久代，逐步改为采用本地内存（Native Memory）来实现方法区的计划了[1]，到了JDK 7的HotSpot，已经把原本放在永久代的字符串常量池、静态变量等移出，而到了JDK 8，终于完全废弃了永久代的概念，改用与JRockit、J9一样在本地内存中实现的元空间（Metaspace）来代替，把JDK 7中永久代还剩余的内容（主要是类型信息）全部移到元空间中。

1、整个永久代有一个 JVM 本身设置的固定大小上限，无法进行调整（也就是受到 JVM 内存的限制），而元空间使用的是本地内存，受本机可用内存的限制，虽然元空间仍旧可能溢出，但是比原来出现的几率会更小。

> 当元空间溢出时会得到如下错误： java.lang.OutOfMemoryError: MetaSpace

你可以使用 `-XX：MaxMetaspaceSize` 标志设置最大元空间大小，默认值为 unlimited，这意味着它只受系统内存的限制。 `-XX：MetaspaceSize` 调整标志定义元空间的初始大小如果未指定此标志，则 Metaspace 将根据运行时的应用程序需求动态地重新调整大小。

2、元空间里面存放的是类的元数据，这样加载多少类的元数据就不由 `MaxPermSize` 控制了，而由系统的实际可用空间来控制，这样能加载的类就更多了。

3、在 JDK8，合并 HotSpot 和 JRockit 的代码时，JRockit 从来没有一个叫永久代的东西，合并之后就没有必要额外的设置这么一个永久代的地方了。

4、永久代会为 GC 带来不必要的复杂度，并且回收效率偏低。

方法区常用参数有哪些?

JDK 1.8 之前永久代还没被彻底移除的时候通常通过下面这些参数来调节方法区大小。

```
-XX:PermSize=N //方法区 (永久代) 初始大小
-XX:MaxPermSize=N //方法区 (永久代) 最大大小,超过这个值将会抛出 OutOfMemoryError 异常:java.lang.OutOfMemoryError: PermGen
```

相对而言，垃圾收集行为在这个区域是比较少出现的，但并非数据进入方法区后就“永久存在”了。

JDK 1.8 的时候，方法区（HotSpot 的永久代）被彻底移除了（JDK1.7 就已经开始了），取而代之是元空间，元空间使用的是本地内存。下面是一些常用参数：

```
-XX:MetaspaceSize=N //设置 Metaspace 的初始（和最小大小）
-XX:MaxMetaspaceSize=N //设置 Metaspace 的最大大小
```

与永久代很大的不同就是，如果不指定大小的话，随着更多类的创建，虚拟机会耗尽所有可用的系统内存。

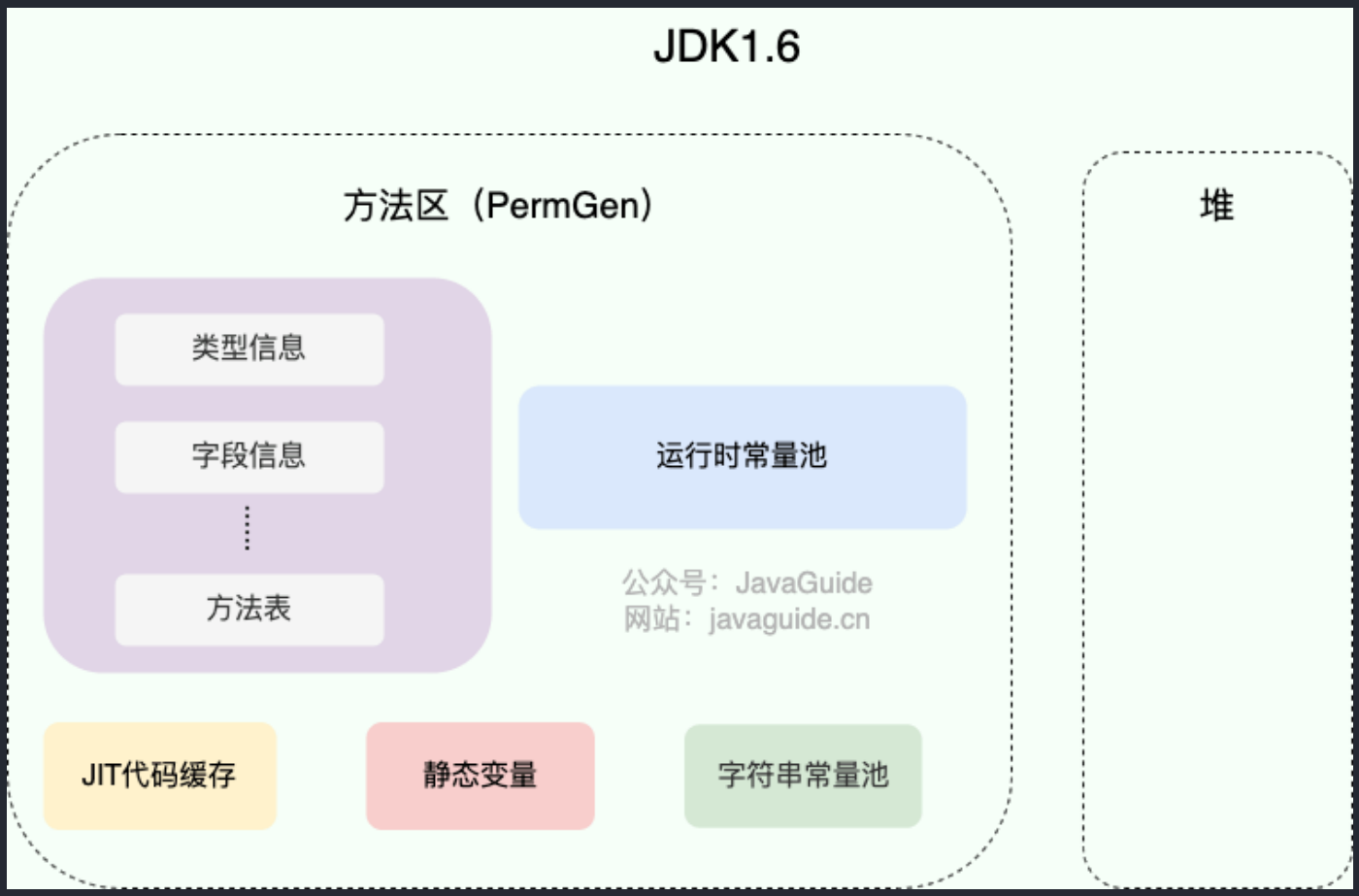
⭐字符串常量池的作用是?

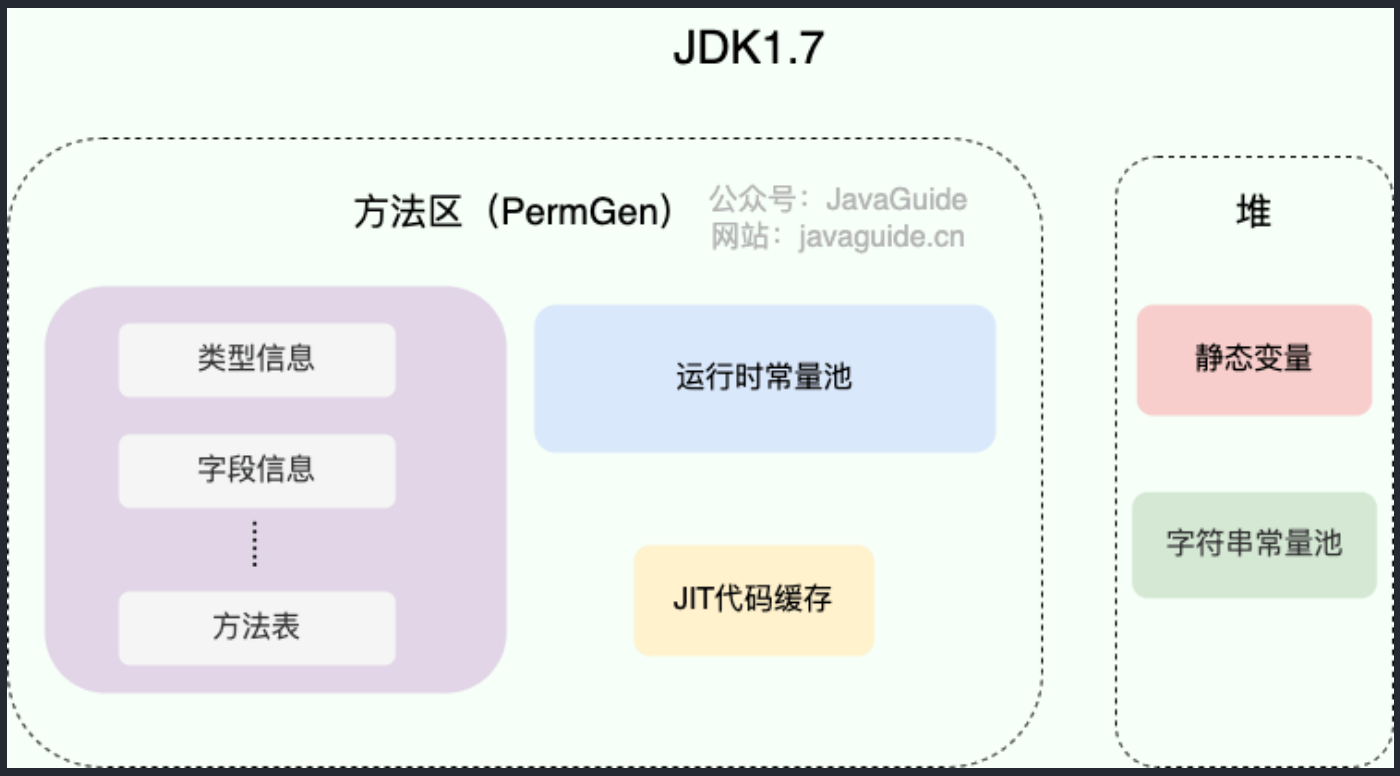
字符串常量池 是 JVM 为了提升性能和减少内存消耗针对字符串（String 类）专门开辟的一块区域，主要目的是为了避免字符串的重复创建。

```
// 在字符串常量池中创建字符串对象 "ab"
// 将字符串对象 "ab" 的引用赋值给给 aa
String aa = "ab";
// 直接返回字符串常量池中字符串对象 "ab"，赋值给引用 bb
String bb = "ab";
System.out.println(aa==bb); // true
```

HotSpot 虚拟机中字符串常量池的实现是 `src/hotspot/share/classfile/stringTable.cpp` ，`StringTable` 可以简单理解为一个固定大小的 `HashTable` ，容量为 `StringTableSize` （可以通过 `-XX:StringTableSize` 参数来设置），保存的是字符串（key）和 字符串对象的引用（value）的映射关系，字符串对象的引用指向堆中的字符串对象。

JDK1.7 之前，字符串常量池存放在永久代。JDK1.7 字符串常量池和静态变量从永久代移动到了 Java 堆中。





JDK 1.7 为什么要将字符串常量池移动到堆中?

主要是因为永久代（方法区实现）的 GC 回收效率太低，只有在整堆收集 (Full GC)的时候才会被执行 GC。Java 程序中通常会有大量的被创建的字符串等待回收，将字符串常量池放到堆中，能够更高效及时地回收字符串内存。

直接内存的作用是?

直接内存是一种特殊的内存缓冲区，并不在 Java 堆或方法区中分配的，而是通过 JNI 的方式在本地内存上分配的。

直接内存并不是虚拟机运行时数据区的一部分，也不是虚拟机规范中定义的内存区域，但是这部分内存也被频繁地使用。而且也可能导致 `OutOfMemoryError` 错误出现。

JDK1.4 中新加入的 **NIO（Non-Blocking I/O，也被称为 New I/O）**，引入了一种基于**通道（Channel）**与**缓存区（Buffer）**的 I/O 方式，它可以直接使用 Native 函数库直接分配堆外内存，然后通过一个存储在 Java 堆中的 DirectByteBuffer 对象作为这块内存的引用进行操作。这样就能在一些场景中显著提高性能，因为**避免了在 Java 堆和 Native 堆之间来回复制数据**。

直接内存的分配不会受到 Java 堆的限制，但是，既然是内存就会受到本机总内存大小以及处理器寻址空间的限制。

类似的概念还有 **堆外内存** 。在一些文章中将直接内存等价于堆外内存，个人觉得不是特别准确。

堆外内存就是把内存对象分配在堆外的内存，这些内存直接受操作系统管理（而不是虚拟机），这样做的结果就是能够在一定程度上减少垃圾回收对应用程序造成的影响。

Java 对象的创建过程

JVM（HotSpot 虚拟机）中对象的创建过程主要分为以下五步：

1. **类加载检查**：虚拟机执行 new 指令时，先检查常量池中对应类的符号引用是否已加载、解析和初始化，未完成则先执行类加载过程。
2. **分配内存**：类加载通过后，根据类加载确定的对象大小从 Java 堆划分内存，分配方式有 “指针碰撞”（适用于堆内存规整，如 Serial/ParNew 收集器）和 “空闲列表”（适用于堆内存不规整，如 CMS 收集器）；为保证线程安全，采用 CAS + 失败重试或 TLAB（线程本地分配缓冲）机制。
3. **初始化零值**：将分配的内存空间（除对象头外）初始化为零值，确保 Java 代码中未赋初始值的实例字段可直接使用对应类型的零值。
4. **设置对象头**：在对象头中记录类元数据信息、哈希码、GC 分代年龄、锁状态等必要信息，具体设置依虚拟机运行状态（如是否启用偏向锁）而定。

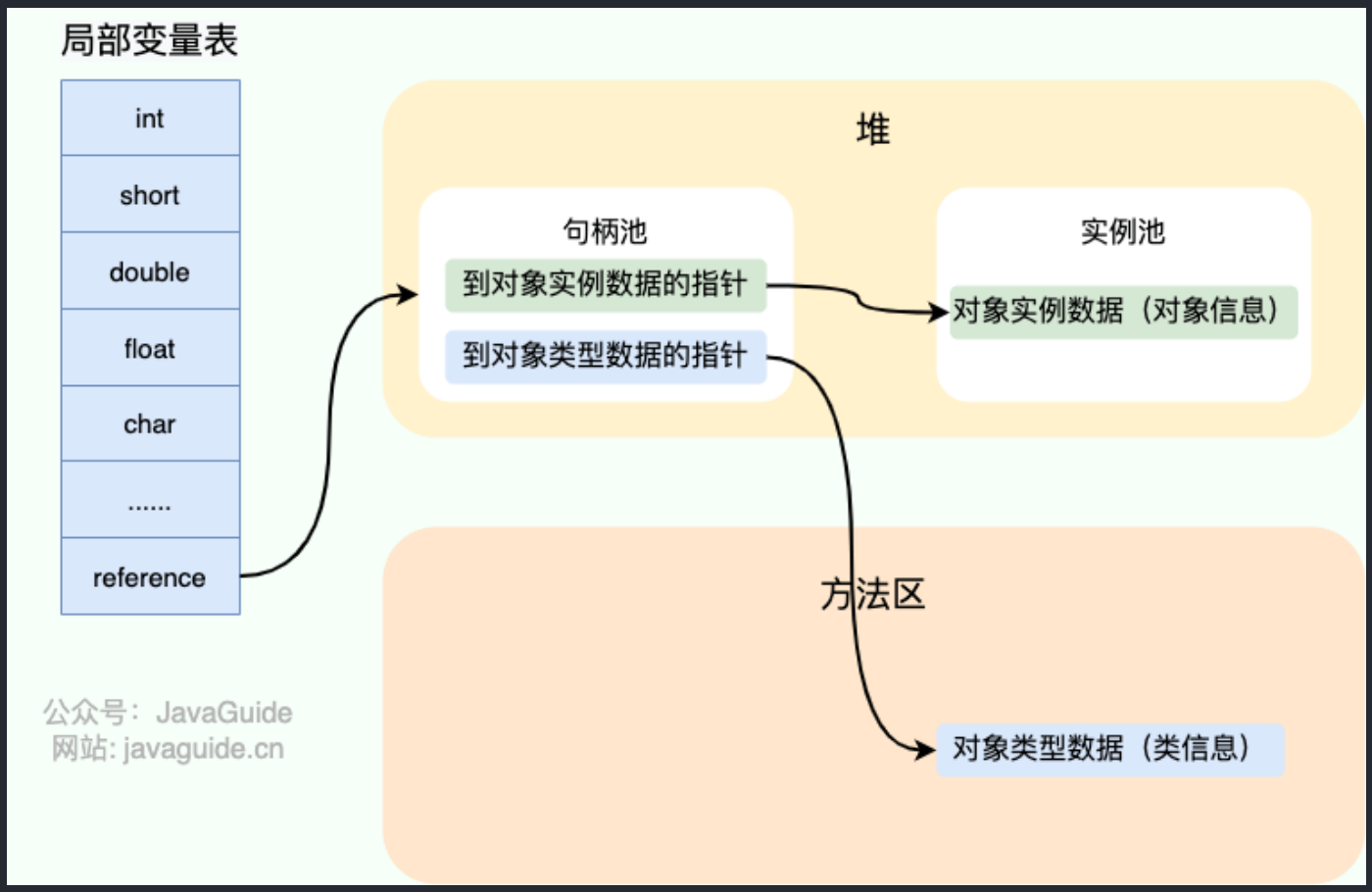
5. **执行 init 方法**：虚拟机视角下对象已创建，但需执行 `<init>` 方法按程序员定义完成初始化，最终生成可用对象。

⭐对象访问定位的方式有哪些?

建立对象就是为了使用对象，我们的 Java 程序通过栈上的 reference 数据来操作堆上的具体对象。对象的访问方式由虚拟机实现而定，目前主流的访问方式有：**使用句柄**、**直接指针**。

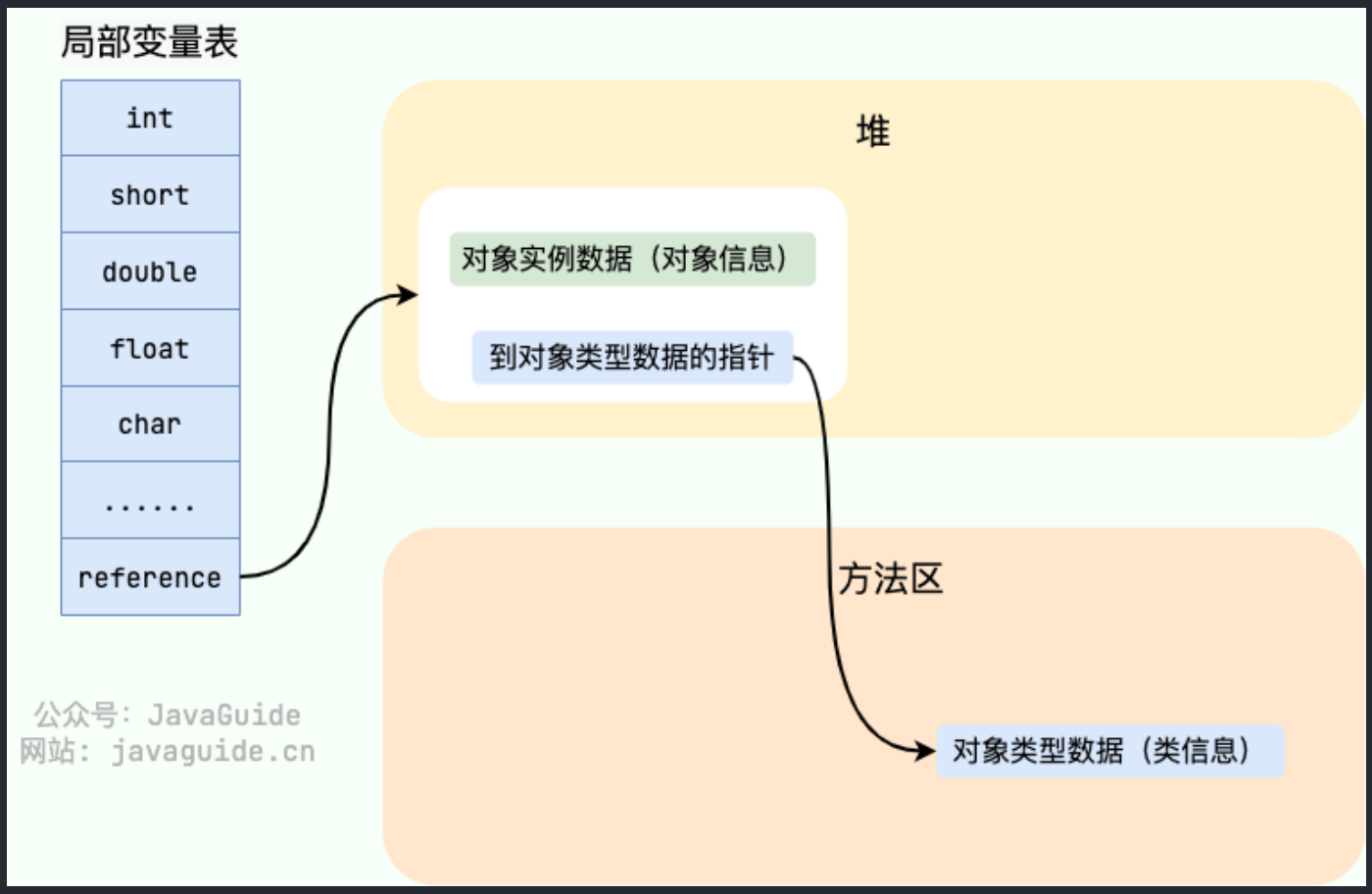
句柄

如果使用句柄的话，那么 Java 堆中将会划分出一块内存来作为句柄池，reference 中存储的就是对象的句柄地址，而句柄中包含了对象实例数据与对象类型数据各自的具体地址信息。



直接指针

如果使用直接指针访问，reference 中存储的直接就是对象的地址。



这两种对象访问方式各有优势。使用句柄来访问的最大好处是 reference 中存储的是稳定的句柄地址，在对象被移动时只会改变句柄中的实例数据指针，而 reference 本身不需要修改。使用直接指针访问方式最大的好处就是速度快，它节省了一次指针定位的时间开销。

HotSpot 虚拟机主要使用的就是这种方式来进行对象访问。

JVM 垃圾回收

⭐如何判断对象是否死亡

堆中几乎放着所有的对象实例，对堆垃圾回收前的第一步就是要判断哪些对象已经死亡（即不能再被任何途径使用的对象）。

引用计数法

给对象中添加一个引用计数器：

- 每当有一个地方引用它，计数器就加 1；
- 当引用失效，计数器就减 1；
- 任何时候计数器为 0 的对象就是不可能再被使用的。

这个方法实现简单，效率高，但是目前主流的虚拟机中并没有选择这个算法来管理内存，其最主要的原因是它很难解决对象之间循环引用的问题。

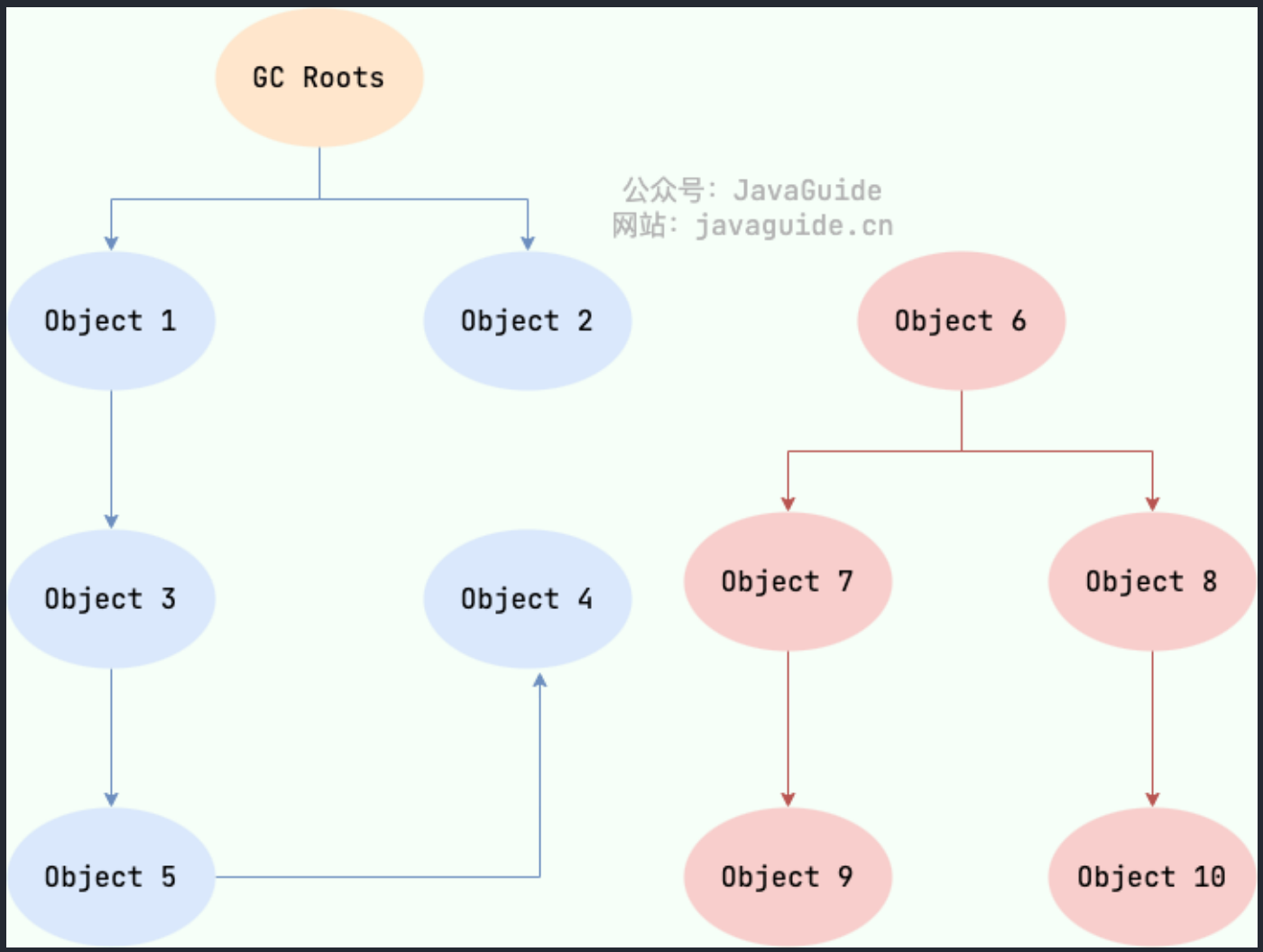
所谓对象之间的相互引用问题，如下面代码所示：除了对象 `objA` 和 `objB` 相互引用着对方之外，这两个对象之间再无任何引用。但是他们因为互相引用对方，导致它们的引用计数器都不为 0，于是引用计数算法无法通知 GC 回收器回收他们。

```
public class ReferenceCountingGc {
    Object instance = null;
    public static void main(String[] args) {
        ReferenceCountingGc objA = new ReferenceCountingGc();
        ReferenceCountingGc objB = new ReferenceCountingGc();
        objA.instance = objB;
        objB.instance = objA;
        objA = null;
        objB = null;
    }
}
```

可达性分析算法

这个算法的基本思想就是通过一系列的称为 **“GC Roots”** 的对象作为起点，从这些节点开始向下搜索，节点所走过的路径称为引用链，当一个对象到 GC Roots 没有任何引用链相连的话，则证明此对象是不可用的，需要被回收。

下图中的 `Object 6 ~ Object 10` 之间虽有引用关系，但它们到 GC Roots 不可达，因此为需要被回收的对象。



哪些对象可以作为 GC Roots 呢?

- 虚拟机栈(栈帧中的局部变量表)中引用的对象
- 本地方法栈(Native 方法)中引用的对象
- 方法区中类静态属性引用的对象
- 方法区中常量引用的对象
- 所有被同步锁持有的对象
- JNI（Java Native Interface）引用的对象

对象可以被回收，就代表一定会被回收吗?

即使在可达性分析法中不可达的对象，也并非是“非死不可”的，这时候它们暂时处于“缓刑阶段”，要真正宣告一个对象死亡，至少要经历两次标记过程；可达性分析法中不可达的对象被第一次标记并且进行一次筛选，筛选的条件是此对象是否有必要执行 `finalize` 方法。当对象没有覆盖 `finalize` 方法，或 `finalize` 方法已经被虚拟机调用过时，虚拟机将这两种情况视为没有必要执行。

被判定为需要执行的对象将会被放在一个队列中进行第二次标记，除非这个对象与引用链上的任何一个对象建立关联，否则就会被真的回收。

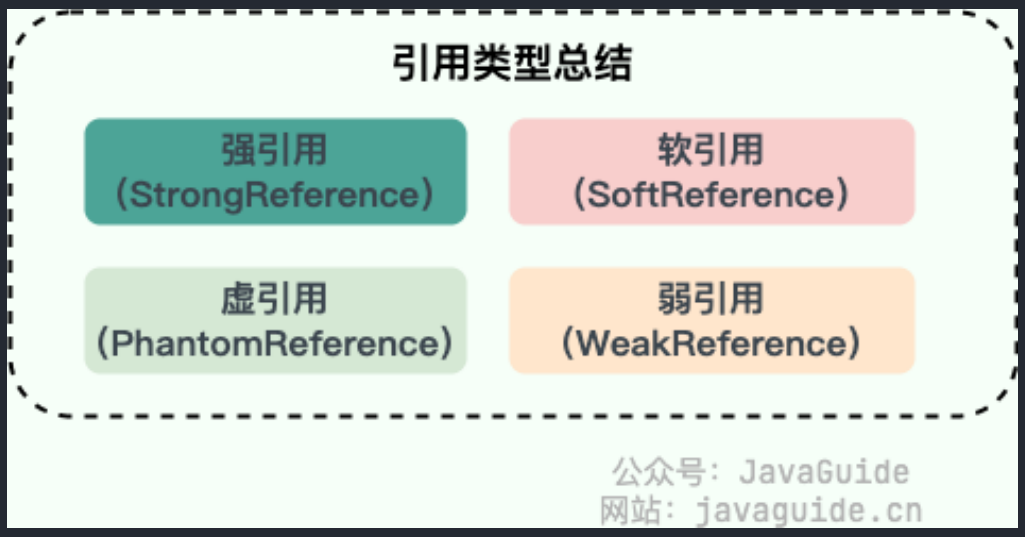
> `Object` 类中的 `finalize` 方法一直被认为是一个糟糕的设计，成为了 Java 语言的负担，影响了 Java 语言的安全和 GC 的性能。JDK9 版本及后续版本中各个类中的 `finalize` 方法会被逐渐弃用移除。忘掉它的存在吧！
>
> 参考：
>
> - JEP 421: Deprecate Finalization for Removal
> - 是时候忘掉 finalize 方法了

常见的引用类型有哪些？

无论是通过引用计数法判断对象引用数量，还是通过可达性分析法判断对象的引用链是否可达，判定对象的存活都与“引用”有关。

JDK1.2 之前，Java 中引用的定义很传统：如果 reference 类型的数据存储的数值代表的是另一块内存的起始地址，就称这块内存代表一个引用。

JDK1.2 以后，Java 对引用的概念进行了扩充，将引用分为强引用、软引用、弱引用、虚引用四种（引用强度逐渐减弱），强引用就是 Java 中普通的对象，而软引用、弱引用、虚引用在 JDK 中定义的类分别是 `SoftReference`、`WeakReference`、`PhantomReference`。



1. 强引用（StrongReference）

强引用实际上就是程序代码中普遍存在的引用赋值，这是使用最普遍的引用，其代码如下

```
String strongReference = new String("abc");
```

如果一个对象具有强引用，那就类似于**必不可少的生活用品**，垃圾回收器绝不会回收它。当内存空间不足，Java 虚拟机宁愿抛出 OutOfMemoryError 错误，使程序异常终止，也不会靠随意回收具有强引用的对象来解决内存不足问题。

2. 软引用（SoftReference）

如果一个对象只具有软引用，那就类似于**可有可无的生活用品**。软引用代码如下

```
// 软引用
String str = new String("abc");
SoftReference<String> softReference = new SoftReference<String>(str);
```

如果内存空间足够，垃圾回收器就不会回收它，如果内存空间不足了，就会回收这些对象的内存。只要垃圾回收器没有回收它，该对象就可以被程序使用。软引用可用来实现内存敏感的高速缓存。

软引用可以和一个引用队列（ReferenceQueue）联合使用，如果软引用所引用的对象被垃圾回收，JAVA 虚拟机就会把这个软引用加入到与之关联的引用队列中。

3. 弱引用（WeakReference）

如果一个对象只具有弱引用，那就类似于**可有可无的生活用品**。弱引用代码如下：

```
String str = new String("abc");
WeakReference<String> weakReference = new WeakReference<>(str);
str = null; //str变成软引用，可以被收集
```

弱引用与软引用的区别在于：只具有弱引用的对象拥有更短暂的生命周期。在垃圾回收器线程扫描它所管辖的内存区域的过程中，一旦发现了只具有弱引用的对象，不管当前内存空间足够与否，都会回收它的内存。不过，由于垃圾回收器是一个优先级很低的线程， 因此不一定会很快发现那些只具有弱引用的对象。

弱引用可以和一个引用队列（ReferenceQueue）联合使用，如果弱引用所引用的对象被垃圾回收，Java 虚拟机就会把这个弱引用加入到与之关联的引用队列中。

4．虚引用（PhantomReference）

"虚引用"顾名思义，就是形同虚设，与其他几种引用都不同，虚引用并不会决定对象的生命周期。如果一个对象仅持有虚引用，那么它就和没有任何引用一样，在任何时候都可能被垃圾回收。虚引用代码如下：

```
String str = new String("abc");
ReferenceQueue queue = new ReferenceQueue();
// 创建虚引用，要求必须与一个引用队列关联
PhantomReference pr = new PhantomReference(str, queue);
```

虚引用主要用来跟踪对象被垃圾回收的活动。

虚引用与软引用和弱引用的一个区别在于： 虚引用必须和引用队列（ReferenceQueue）联合使用。当垃圾回收器准备回收一个对象时，如果发现它还有虚引用，就会在回收对象的内存之前，把这个虚引用加入到与之关联的引用队列中。程序可以通过判断引用队列中是否已经加入了虚引用，来了解被引用的对象是否将要被垃圾回收。程序如果发现某个虚引用已经被加入到引用队列，那么就可以在所引用的对象的内存被回收之前采取必要的行动。

特别注意，在程序设计中一般很少使用弱引用与虚引用，使用软引用的情况较多，这是因为**软引用可以加速 JVM 对垃圾内存的回收速度，可以维护系统的运行安全，防止内存溢出（OutOfMemory）等问题的产生。**

如何判断一个类是无用的类?

方法区主要回收的是无用的类，那么如何判断一个类是无用的类的呢?

判定一个常量是否是“废弃常量”比较简单，而要判定一个类是否是“无用的类”的条件则相对苛刻许多。类需要同时满足下面 3 个条件才能算是 **“无用的类”**：

- 该类所有的实例都已经被回收，也就是 Java 堆中不存在该类的任何实例。
- 加载该类的 `ClassLoader` 已经被回收。
- 该类对应的 `java.lang.Class` 对象没有在任何地方被引用，无法在任何地方通过反射访问该类的方法。

虚拟机可以对满足上述 3 个条件的无用类进行回收，这里说的仅仅是“可以”，而并不是和对象一样不使用了就会必然被回收。

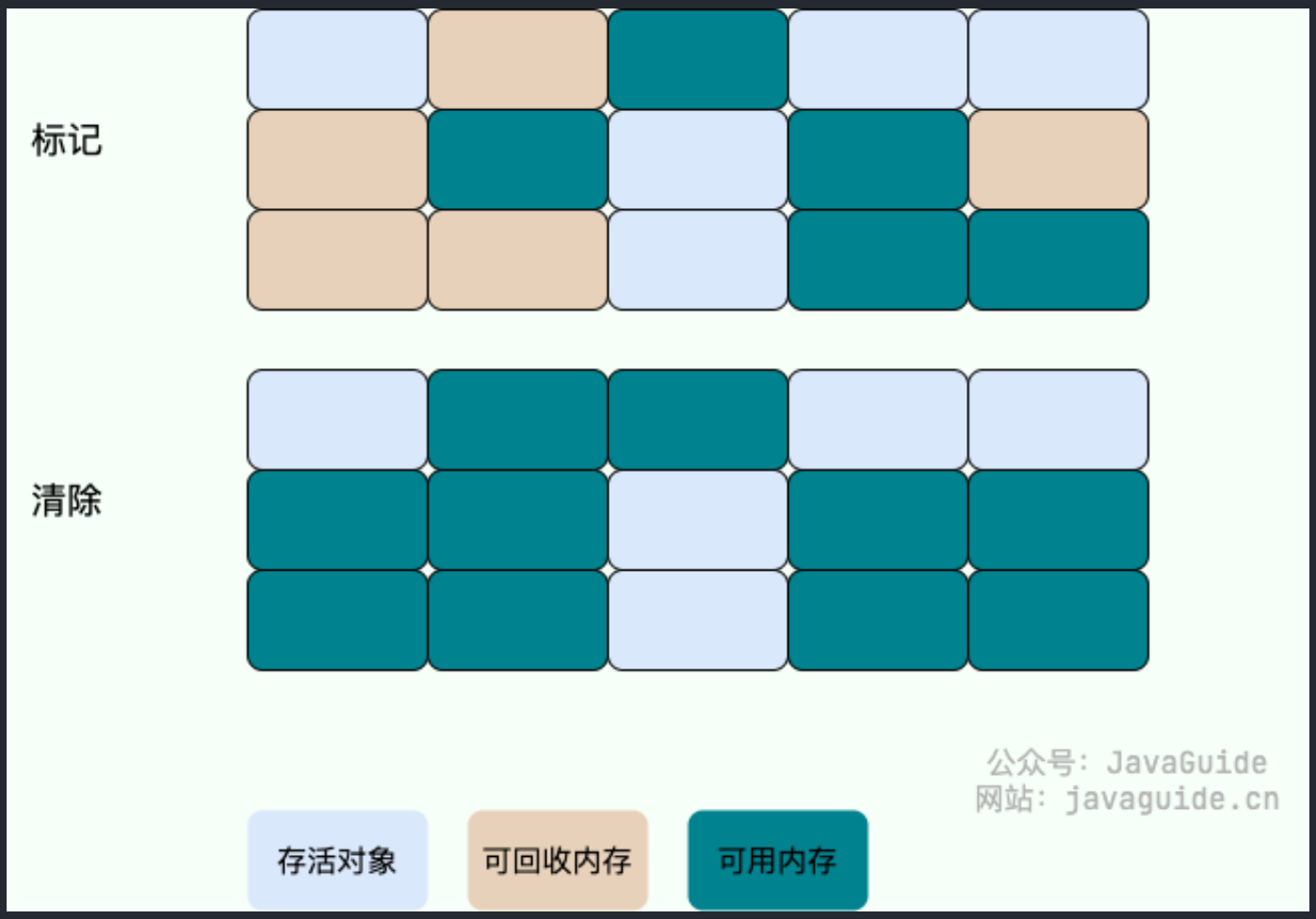
⭐垃圾回收算法有哪些?

标记-清除算法

标记-清除（Mark-and-Sweep）算法分为“标记（Mark）”和“清除（Sweep）”阶段：首先标记出所有不需要回收的对象，在标记完成后统一回收掉所有没有被标记的对象。

它是最基础的收集算法，后续的算法都是对其不足进行改进得到。这种垃圾收集算法会带来两个明显的问题：

1. **效率问题**：标记和清除两个过程效率都不高。
2. **空间问题**：标记清除后会产生大量不连续的内存碎片。



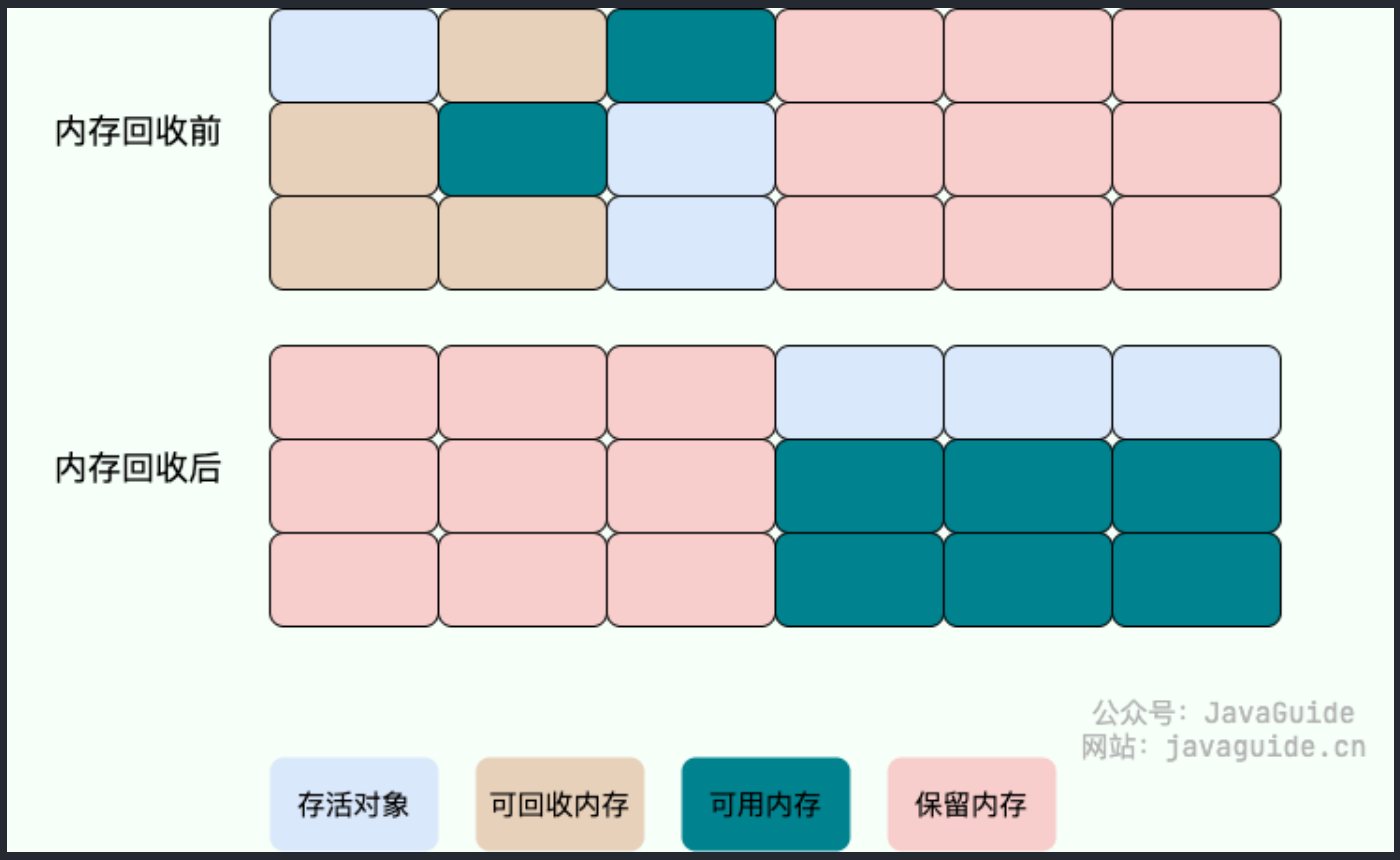
关于具体是标记可回收对象还是不可回收对象，众说纷纭，两种说法其实都没问题，我个人更倾向于是前者。

如果按照前者的理解，整个标记-清除过程大致是这样的：

1. 当一个对象被创建时，给一个标记位，假设为 0 (false)；
2. 在标记阶段，我们将所有可达对象（或用户可以引用的对象）的标记位设置为 1 (true)；
3. 扫描阶段清除的就是标记位为 0 (false)的对象。

复制算法

为了解决标记-清除算法的效率和内存碎片问题，复制（Copying）收集算法出现了。它可以将内存分为大小相同的两块，每次使用其中的一块。当这一块的内存使用完后，就将还存活的对象复制到另一块去，然后再把使用的空间一次清理掉。这样就使每次的内存回收都是对内存区间的一半进行回收。

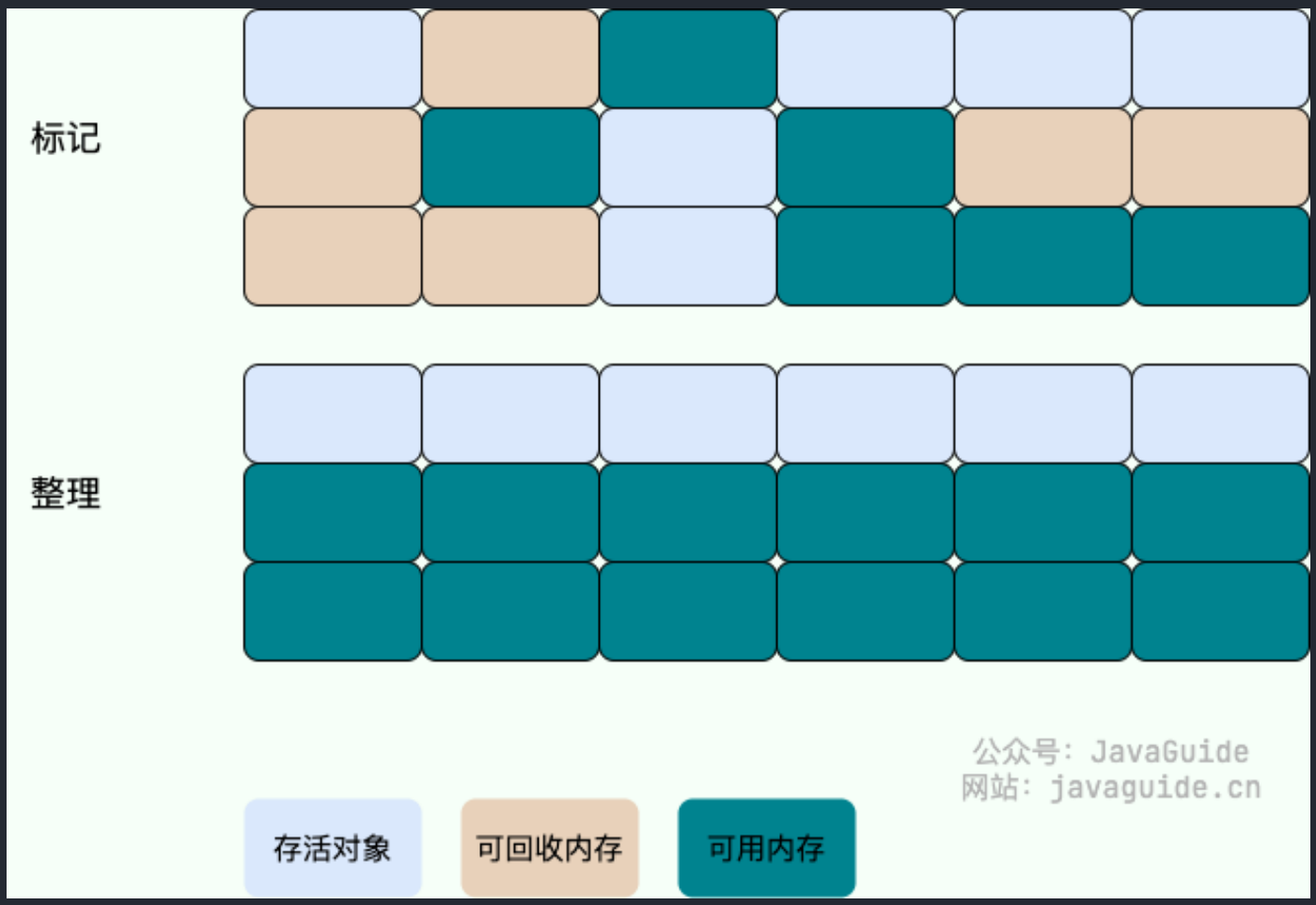


虽然改进了标记-清除算法，但依然存在下面这些问题：

- **可用内存变小**：可用内存缩小为原来的一半。
- **不适合老年代**：如果存活对象数量比较大，复制性能会变得很差。

标记-整理算法

标记-整理（Mark-and-Compact）算法是根据老年代的特点提出的一种标记算法，标记过程仍然与“标记-清除”算法一样，但后续步骤不是直接对可回收对象回收，而是让所有存活的对象向一端移动，然后直接清理掉端边界以外的内存。



由于多了整理这一步，因此效率也不高，适合老年代这种垃圾回收频率不是很高的场景。

分代收集算法

当前虚拟机的垃圾收集都采用分代收集算法，这种算法没有什么新的思想，只是根据对象存活周期的不同将内存分为几块。一般将 Java 堆分为新生代和老年代，这样我们就可以根据各个年代的特点选择合适的垃圾收集算法。

比如在新生代中，每次收集都会有大量对象死去，所以可以选择”标记-复制“算法，只需要付出少量对象的复制成本就可以完成每次垃圾收集。而老年代的对象存活几率是比较高的，而且没有额外的空间对它进行分配担保，所以我们必须选择“标记-清除”或“标记-整理”算法进行垃圾收集。

延伸面试问题： HotSpot 为什么要分为新生代和老年代？

根据上面的对分代收集算法的介绍回答。

⭐JDK 1.8 的默认垃圾回收器是？JDK1.9 之后呢？

- **JDK 1.8 默认垃圾回收器**：Parallel Scanvenge（新生代）+ Parallel Old（老年代）。 这个组合也被称为 Parallel GC 或 Throughput GC，侧重于吞吐量。
- **JDK 1.9 及以后默认垃圾回收器**：G1 GC (Garbage-First Garbage Collector)。 G1 GC 是一个更现代化的垃圾回收器，旨在平衡吞吐量和停顿时间，尤其适用于堆内存较大的应用。

⭐G1 垃圾回收的过程

G1（Garbage-First）垃圾收集器在 JDK 7 中首次引入，作为一种试验性的垃圾收集器。到了 JDK 8，G1 得到了进一步的完善和改进，功能基本已经完全实现，成为一个稳定、可用于生产环境的垃圾收集器。

G1 收集器的运作大致分为以下几个步骤：

- **初始标记**： 短暂停顿（Stop-The-World，STW），标记从 GC Roots 可直接引用的对象，即标记所有直接可达的活跃对象
- **并发标记**：与应用并发运行，标记所有可达对象。 这一阶段可能持续较长时间，取决于堆的大小和对象的数量。
- **最终标记**： 短暂停顿（STW），处理并发标记阶段结束后残留的少量未处理的引用变更。
- **筛选回收**：根据标记结果，选择回收价值高的区域，复制存活对象到新区域，回收旧区域内存。这一阶段包含一个或多个停顿（STW），具体取决于回收的复杂度。

G1 收集器运行示意图

G1 收集器在后台维护了一个优先列表，每次根据允许的收集时间，优先选择回收价值最大的 Region(这也就是它的名字 Garbage-First 的由来) 。这种使用 Region 划分内存空间以及有优先级的区域回收方式，保证了 G1 收集器在有限时间内可以尽可能高的收集效率（把内存化整为零）。

⭐ZGC 有哪些改进？

与 CMS、ParNew 和 G1 类似，ZGC 也采用标记-复制算法，不过 ZGC 对该算法做了重大改进。

ZGC 可以将暂停时间控制在几毫秒以内，且暂停时间不受堆内存大小的影响，出现 Stop The World 的情况会更少，但代价是牺牲了一些吞吐量。ZGC 最大支持 16TB 的堆内存。

ZGC 在 Java11 中引入，处于试验阶段。经过多个版本的迭代，不断的完善和修复问题，ZGC 在 Java15 已经可以正式使用了。

不过，默认的垃圾回收器依然是 G1。你可以通过下面的参数启用 ZGC：

```
java -XX:+UseZGC className
```

在 Java21 中，引入了分代 ZGC，暂停时间可以缩短到 1 毫秒以内。

你可以通过下面的参数启用分代 ZGC：

```
java -XX:+UseZGC -XX:+ZGenerational className
```

关于 ZGC 收集器的详细介绍推荐看看这几篇文章：

- 从历代 GC 算法角度剖析 ZGC - 京东技术
- 新一代垃圾回收器 ZGC 的探索与实践 - 美团技术团队
- 极致八股文之 JVM 垃圾回收器 G1&ZGC 详解 - 阿里云开发者

⭐双亲委派模型

双亲委派模型指的是?

类加载器有很多种，当我们想要加载一个类的时候，具体是哪个类加载器加载呢？这就需要提到双亲委派模型了。

根据官网介绍：

> The ClassLoader class uses a delegation model to search for classes and resources. Each instance of ClassLoader has an associated parent class loader. When requested to find a class or resource, a ClassLoader instance will delegate the search for the class or resource to its parent class loader before attempting to find the class or resource itself. The virtual machine's built-in class loader, called the "bootstrap class loader", does not itself have a parent but may serve as the parent of a ClassLoader instance.

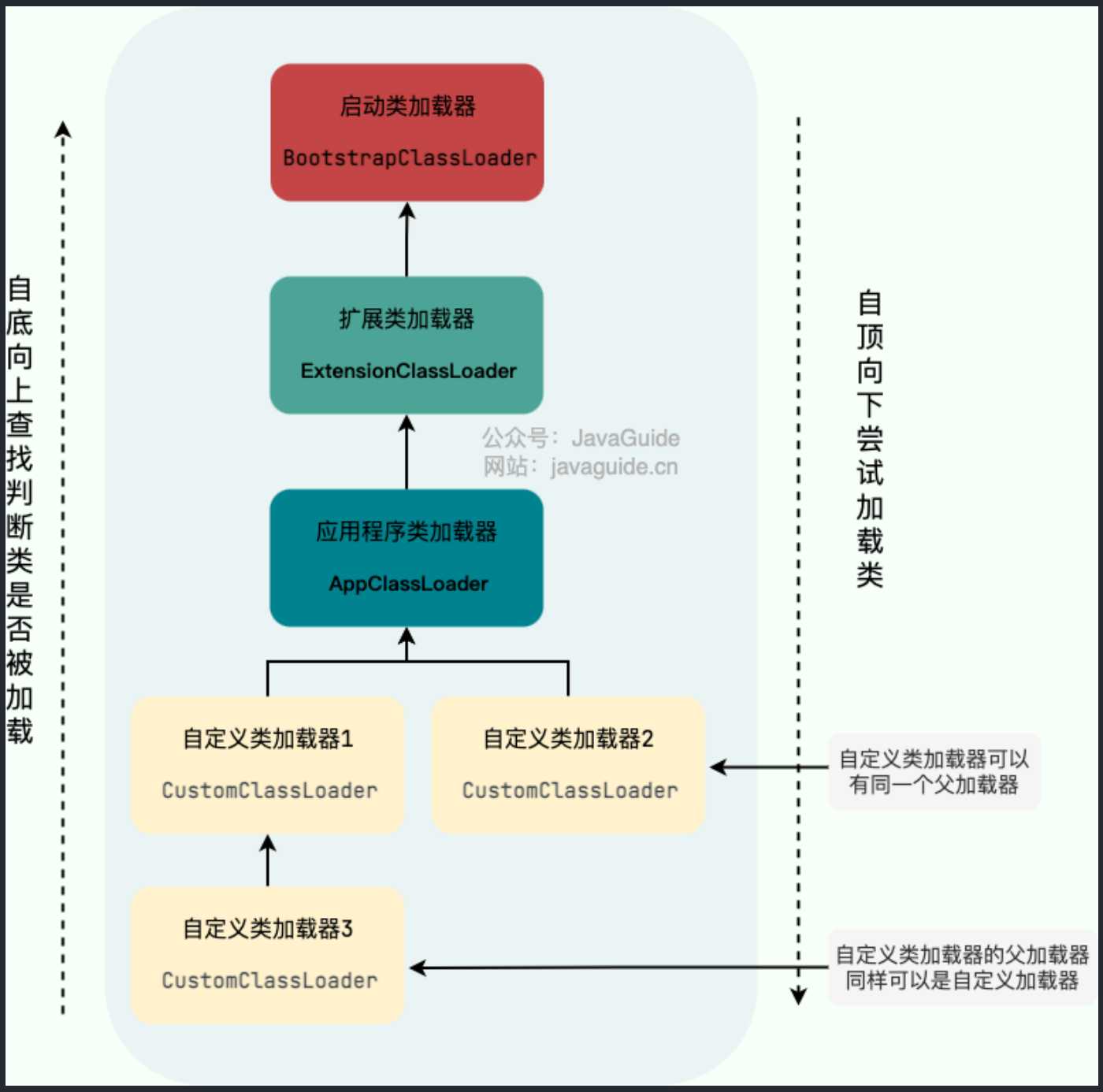
翻译过来大概的意思是：

> `ClassLoader` 类使用委托模型来搜索类和资源。每个 `ClassLoader` 实例都有一个相关的父类加载器。需要查找类或资源时， `ClassLoader` 实例会在试图亲自查找类或资源之前，将搜索类或资源的任务委托给其父类加载器。
> 虚拟机中被称为 "bootstrap class loader"的内置类加载器本身没有父类加载器，但是可以作为 `ClassLoader` 实例的父类加载器。

从上面的介绍可以看出：

- `ClassLoader` 类使用委托模型来搜索类和资源。
- 双亲委派模型要求除了顶层的启动类加载器外，其余的类加载器都应有自己的父类加载器。
- `ClassLoader` 实例会在试图亲自查找类或资源之前，将搜索类或资源的任务委托给其父类加载器。

下图展示的各种类加载器之间的层次关系被称为类加载器的“**双亲委派模型(Parents Delegation Model)**”。



注意 ⚠️：双亲委派模型并不是一种强制性的约束，只是 JDK 官方推荐的一种方式。如果我们因为某些特殊需求想要打破双亲委派模型，也是可以的，后文会介绍具体的方法。

其实这个双亲翻译的容易让别人误解，我们一般理解的双亲都是父母，这里的双亲更多地表达的是“父母这一辈”的人而已，并不是说真的有一个 `MotherClassLoader` 和一个 `FatherClassLoader` 。个人觉得翻译成单亲委派模型更好一些，不过，国内既然翻译成了双亲委派模型并流传了，按照这个来也没问题，不要被误解了就好。

另外，类加载器之间的父子关系一般不是以继承的关系来实现的，而是通常使用组合关系来复用父加载器的代码。

```
public abstract class ClassLoader {
  ...
  // 组合
  private final ClassLoader parent;
  protected ClassLoader(ClassLoader parent) {
       this(checkCreateClassLoader(), parent);
  }
  ...
}
```

在面向对象编程中，有一条非常经典的设计原则：**组合优于继承，多用组合少用继承。**

如何打破打破双亲委派模型?

定义加载器的话，需要继承 `ClassLoader` 。如果我们不想打破双亲委派模型，就重写 `ClassLoader` 类中的 `findClass()` 方法即可，无法被父类加载器加载的类最终会通过这个方法被加载。但是，如果想打破双亲委派模型则需要重写 `loadClass()` 方法。

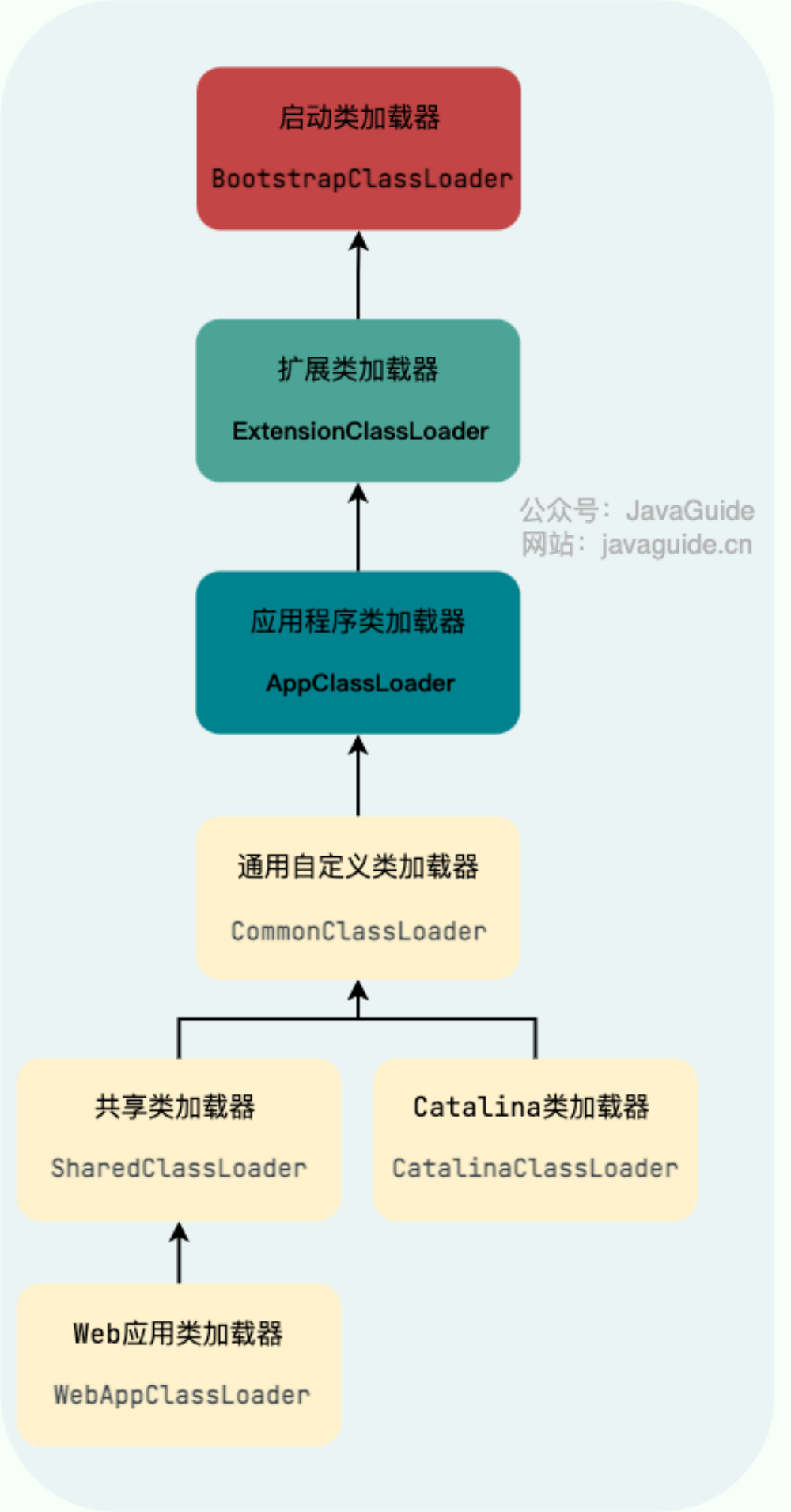
为什么是重写 `loadClass()` 方法打破双亲委派模型呢？双亲委派模型的执行流程已经解释了：

> 类加载器在进行类加载的时候，它首先不会自己去尝试加载这个类，而是把这个请求委派给父类加载器去完成（调用父加载器 `loadClass()` 方法来加载类）。

重写 `loadClass()` 方法之后，我们就可以改变传统双亲委派模型的执行流程。例如，子类加载器可以在委派给父类加载器之前，先自己尝试加载这个类，或者在父类加载器返回之后，再尝试从其他地方加载这个类。具体的规则由我们自己实现，根据项目需求定制化。

我们比较熟悉的 Tomcat 服务器为了能够优先加载 Web 应用目录下的类，然后再加载其他目录下的类，就自定义了类加载器 `WebAppClassLoader` 来打破双亲委托机制。这也是 Tomcat 下 Web 应用之间的类实现隔离的具体原理。

Tomcat 的类加载器的层次结构如下：


启动类加载器
BootstrapClassLoader
扩展类加载器
ExtensionClassLoader
公众号：JavaGuide
网站：javaguide.cn
应用程序类加载器
AppClassLoader
通用自定义类加载器
CommonClassLoader
共享类加载器
SharedClassLoader
Catalina类加载器
CatalinaClassLoader
Web应用类加载器
WebAppClassLoader

Tomcat 这四个自定义的类加载器对应的目录如下：

- `CommonClassLoader` 对应 `<Tomcat>/common/*`
- `CatalinaClassLoader` 对应 `<Tomcat >/server/*`
- `SharedClassLoader` 对应 `<Tomcat >/shared/*`
- `WebAppClassloader` 对应 `<Tomcat >/webapps/<app>/WEB-INF/*`

从图中的委派关系中可以看出：

- `CommonClassLoader` 作为 `CatalinaClassLoader` 和 `SharedClassLoader` 的父加载器。`CommonClassLoader` 能加载的类都可以被 `CatalinaClassLoader` 和 `SharedClassLoader` 使用。因此，`CommonClassLoader` 是为了实现公共类库（可以被所有 Web 应用和 Tomcat 内部组件使用的类库）的共享和隔离。
- `CatalinaClassLoader` 和 `SharedClassLoader` 能加载的类则与对方相互隔离。`CatalinaClassLoader` 用于加载 Tomcat 自身的类，为了隔离 Tomcat 本身的类和 Web 应用的类。`SharedClassLoader` 作为 `WebAppClassLoader` 的父加载器，专门来加载 Web 应用之间共享的类比如 Spring、Mybatis。
- 每个 Web 应用都会创建一个单独的 `WebAppClassLoader`，并在启动 Web 应用的线程里设置线程线程上下文类加载器为 `WebAppClassLoader`。各个 `WebAppClassLoader` 实例之间相互隔离，进而实现 Web 应用之间的类隔。

单纯依靠自定义类加载器没办法满足某些场景的要求，例如，有些情况下，高层的类加载器需要加载低层的加载器才能加载的类。

比如，SPI 中，SPI 的接口（如 `java.sql.Driver`）是由 Java 核心库提供的，由 `BootstrapClassLoader` 加载。而 SPI 的实现（如 `com.mysql.cj.jdbc.Driver`）是由第三方供应商提供的，它们是由应用程序类加载器或者自定义类加载器来加载的。默认情况下，一个类及其依赖类由同一个类加载器加载。所以，加载 SPI 的接口的类加载器（`BootstrapClassLoader`）也会用来加载 SPI 的实现。按照双亲委派模型，`BootstrapClassLoader` 是无法找到 SPI 的实现类的，因为它无法委托给子类加载器去尝试加载。

再比如，假设我们的项目中有 Spring 的 jar 包，由于其是 Web 应用之间共享的，因此会由 `SharedClassLoader` 加载（Web 服务器是 Tomcat）。我们项目中有一些用到了 Spring 的业务类，比如实现了 Spring 提供的接口、用到了 Spring 提供的注解。所以，加载 Spring 的类加载器（也就是 `SharedClassLoader`）也会用来加载这些业务类。但是业务类在 Web 应用目录下，不在 `SharedClassLoader` 的加载路径下，所以 `SharedClassLoader` 无法找到业务类，也就无法加载它们。

如何解决这个问题呢？ 这个时候就需要用到 **线程上下文类加载器（`ThreadContextClassLoader`）**了。

拿 Spring 这个例子来说，当 Spring 需要加载业务类的时候，它不是用自己的类加载器，而是用当前线程的上下文类加载器。还记得我上面说的吗？每个 Web 应用都会创建一个单独的 `WebAppClassLoader`，并在启动 Web 应用的线程里设置线程线程上下文类加载器为 `WebAppClassLoader`。这样就可以让高层的类加载器（`SharedClassLoader`）借助子类加载器（`WebAppClassLoader`）来加载业务类，破坏了 Java 的类加载委托机制，让应用逆向使用类加载器。

线程上下文类加载器的原理是将一个类加载器保存在线程私有数据里，跟线程绑定，然后在需要的时候取出来使用。这个类加载器通常是由应用程序或者容器（如 Tomcat）设置的。

`Java.lang.Thread` 中的 `getContextClassLoader()` 和 `setContextClassLoader(ClassLoader cl)` 分别用来获取和设置线程的上下文类加载器。如果没有通过 `setContextClassLoader(ClassLoader cl)` 进行设置的话，线程将继承其父线程的上下文类加载器。

Spring 获取线程线程上下文类加载器的代码如下：

```
cl = Thread.currentThread().getContextClassLoader();
```

感兴趣的小伙伴可以自行深入研究一下 Tomcat 打破双亲委派模型的原理，推荐资料：《深入拆解 Tomcat & Jetty》。

⭐问题排查

你知道哪些 Java 性能优化和问题排查工具？

JDK 自带的可视化分析工具：

- **JConsole**：基于 JMX 的可视化监视、管理工具，可以用于查看应用程序的运行概况、内存、线程、类、VM 概括、MBean 等信息。
- **VisualVM**：基于 NetBeans 平台开发，具备了插件扩展功能的特性。利用它不仅能够监控服务的 CPU、内存、线程、类等信息，还可以捕获有关 JVM 软件实例的数据，并将该数据保存到本地系统，以供后期查看或与其他用户共享。根据《深入理解 Java 虚拟机》介绍：“VisualVM 的性能分析功能甚至比起 JProfiler、YourKit 等专业且收费的 Profiling 工具都不会逊色多少，而且 VisualVM 还有一个很大的优点：不需要被监视的程序基于特殊 Agent 运行，因此他对应

用程序的实际性能的影响很小，使得他可以直接应用在生产环境中。这个优点是 JProfiler、YourKit 等工具无法与之媲美的”。

JDK 自带的命令行工具：

- `jps` (JVM Process Status）：类似 UNIX 的 `ps` 命令。用于查看所有 Java 进程的启动类、传入参数和 Java 虚拟机参数等信息；
- `jstat` （JVM Statistics Monitoring Tool）：用于收集 HotSpot 虚拟机各方面的运行数据;
- `jinfo` (Configuration Info for Java) : Configuration Info for Java,显示虚拟机配置信息;
- `jmap` (Memory Map for Java) : 生成堆转储快照;
- `jhat` (JVM Heap Dump Browser) : 用于分析 heapdump 文件，它会建立一个 HTTP/HTML 服务器，让用户可以在浏览器上查看分析结果。JDK9 移除了 jhat；
- `jstack` (Stack Trace for Java) : 生成虚拟机当前时刻的线程快照，线程快照就是当前虚拟机内每一条线程正在执行的方法堆栈的集合。

第三方工具：

- **MAT**：一款功能强大的 Java 堆内存分析器，可以用于查找内存泄漏以及查看内存消耗情况，用户可以利用 VisualVM 或者是 `jmap` 命令生产堆文件，然后导入工具中进行分析。
- **GCeasy**：一款在线的 GC 日志分析器，使用起来非常方便，用户可以通过它的 Web 网站导入 GC 日志，实时进行内存泄漏检测、GC 暂停原因分析、JVM 配置建议优化等功能。网站地址：https://gceasy.io/ 。
- **GCViewer**：一款非常强大的 GC 日志可视化分析工具，功能强大而且完全免费。
- **JProfiler**：一款商用的性能分析利器，功能强大，但需要付费使用。 它提供更深入的性能分析功能，例如方法调用分析、内存分配分析等。
- **Arthas**：阿里开源的一款线上监控诊断工具，可以查看应用负载、内存、gc、线程等信息。

如何查看服务器上运行的 Java 进程?

JDK 自带的 `jps` (JVM Process Status) 命令专门用于列出当前用户下所有正在运行的 JVM 实例。

`jps` 的基础用法和几个核心参数如下：

- `jps` ：这是最基础的用法，它会列出 Java 进程的 **LVMID**（本地虚拟机唯一 ID，通常就是操作系统的进程号 PID）和**主类名**（或 Jar 包名）。

- `jps -l`：这是我最常用的参数之一。它会输出主类的**完整包名**，或者如果应用是通过 Jar 包运行的，会输出 Jar 包的**完整路径**。这在同一台机器上部署了多个来自不同项目的 Java 应用时，能非常清晰地区分它们。
- `jps -v`：这个参数也非常实用，尤其是在排查配置问题时。它会显示传递给 JVM 的参数，例如 `-Xmx`、`-Xms`、`-XX:+UseG1GC` 等。通过它，我可以快速确认应用的内存配置、GC 策略等是否符合预期。
- `jps -m`：这个参数用于查看传递给主函数 `main()` 的参数。当我们需要确认程序启动时传入的业务参数是否正确时，它非常有用。

在某些情况下，`jps` 命令可能无法满足需求，这时我会采用标准的操作系统命令：

1. **权限问题**：jps 默认只能看到由**当前用户**启动的 Java 进程。如果需要查看服务器上所有用户（如 root 或其他业务用户）的 Java 进程，jps 就会受限。
2. **环境问题**：在一些极简的生产环境或 Docker 容器中，可能只安装了 JRE 而没有完整的 JDK，此时 jps 命令可能不存在。

在这些情况下，我会使用 ps 命令来查找，例如：

```
# 列出所有进程，然后通过 grep 过滤出包含 "java" 关键字的进程
ps -ef | grep java
```

堆内存相关的 JVM 参数有哪些?

堆内存大小控制：

1. `-Xms`：设置 JVM 初始堆内存大小（如 `-Xms512m` 表示初始堆为 512MB）。
2. `-Xmx`：设置 JVM 最大堆内存大小（如 `-Xmx1g` 表示最大堆为 1GB）。

在生产环境中，强烈建议将 `-Xms` 和 `-Xmx` 设置为相同的值。这样做可以避免 JVM 在运行时根据负载情况动态地收缩和扩展堆内存，这个过程会引发不必要的 Full GC 和性能抖动，从而提高服务的稳定性和响应速度。

新生代与老年代：

1. **`-Xmn`**：这是最直接控制新生代大小的方式，优先级高于 `-XX:NewRatio`。设置后，老年代的大小就是 `-Xmx` 减去 `-Xmn`。当我们对应用的对象生命周期有明确的判断时（例如，有大量的短生命周期对象），可以直接给新生代一个合适的大小，以达到更好的 GC 性能。
2. **`-XX:NewRatio`**：这是另一种调节新生代大小的方式，默认值为 2，表示老年代:新生代 = 2:1。因此，新生代默认占整个堆的 1/3。如果设置为 3，则新生代占堆的 1/4。通常在 `-Xmn` 和 `-XX:NewRatio` 中选择一个使用即可。
3. **`-XX:SurvivorRatio`**：设置新生代中 Eden 区与单个 Survivor 区的比例。默认值为 8，表示 Eden : From Survivor : To Survivor = 8:1:1。所以 Eden 区占整个新生代的 8/10。这个比例会影响对象能否在新生代中“存活”足够长的时间。如果 Survivor 区太小（即 `-XX:SurvivorRatio` 值过大），Minor GC 后存活的对象可能因为放不下而被迫提前进入老年代，增加 Full GC 的压力。

堆内存溢出相关参数：

1. **`-XX:+HeapDumpOnOutOfMemoryError`**：当发生 `OutOfMemoryError`（OOM）时，自动生成堆转储文件（`.hprof`），记录堆内存对象状态。
2. **`-XX:HeapDumpPath`**：指定 OOM 时堆转储文件的保存路径（如 `-XX:HeapDumpPath=/logs/heapdump.hprof`），默认生成在程序运行目录。

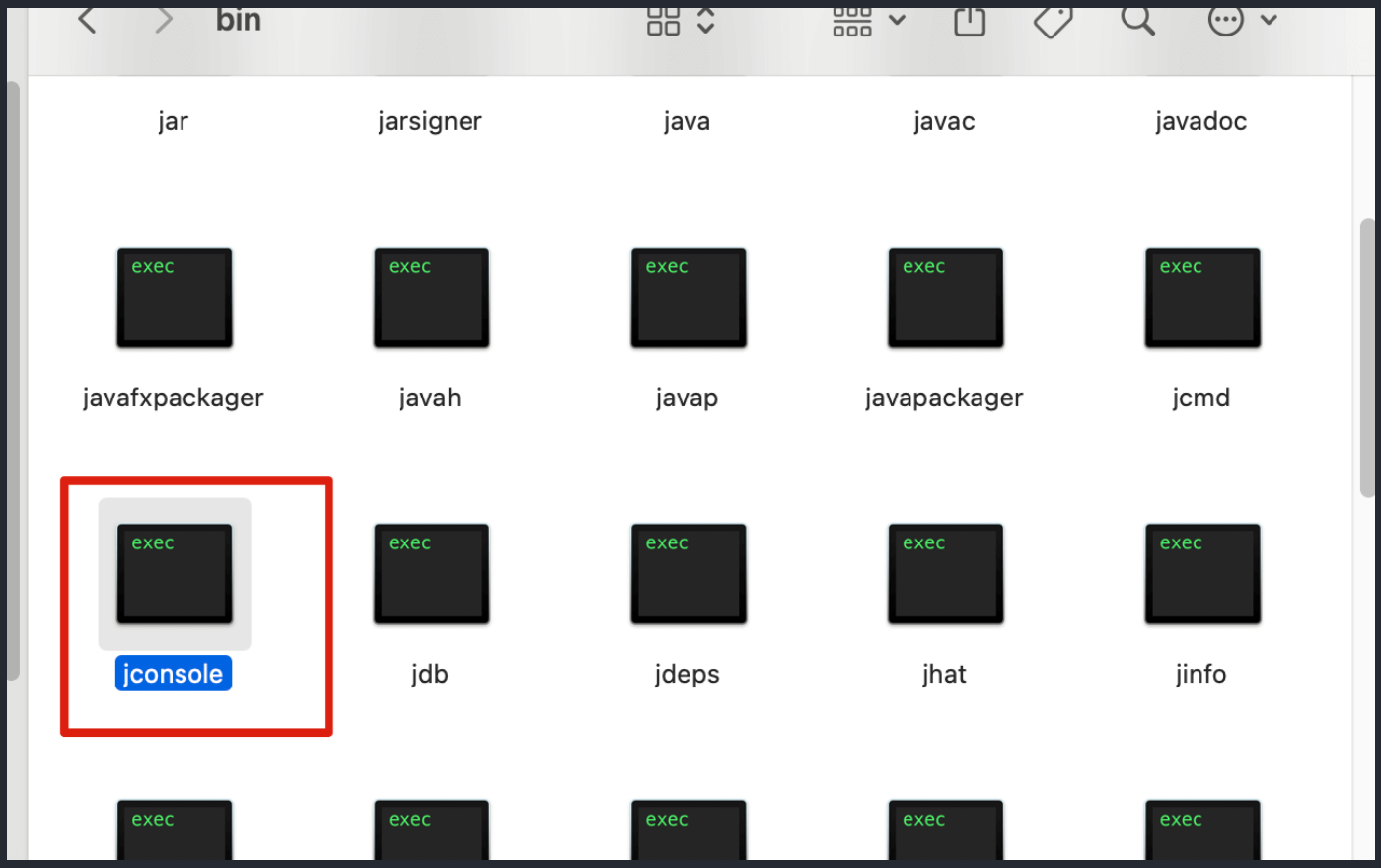
最重要的 JVM 参数可以参考这篇文章：最重要的 JVM 参数总结。

如何检测死锁?

- 使用 `jmap`、`jstack` 等命令查看 JVM 线程栈和堆内存的情况。如果有死锁，`jstack` 的输出中通常会有 `Found one Java-level deadlock:` 的字样，后面会跟着死锁相关的线程信息。另外，实际项目中还可以搭配使用 `top`、`df`、`free` 等命令查看操作系统的基本情况，出现死锁可能会导致 CPU、内存等资源消耗过高。
- 采用 VisualVM、JConsole 等工具进行排查。

这里以 JConsole 工具为例进行演示。

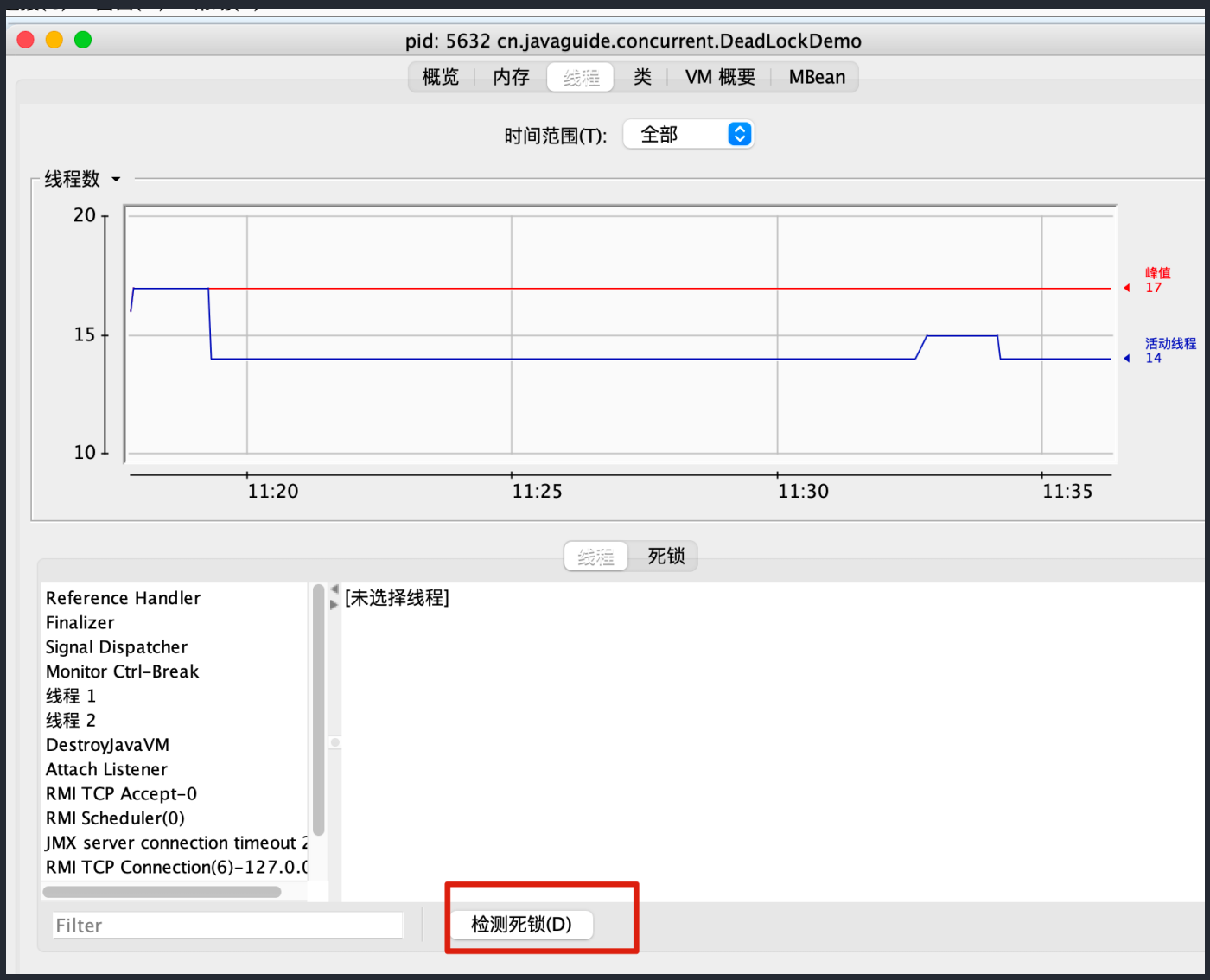
首先，我们要找到 JDK 的 bin 目录，找到 jconsole 并双击打开。

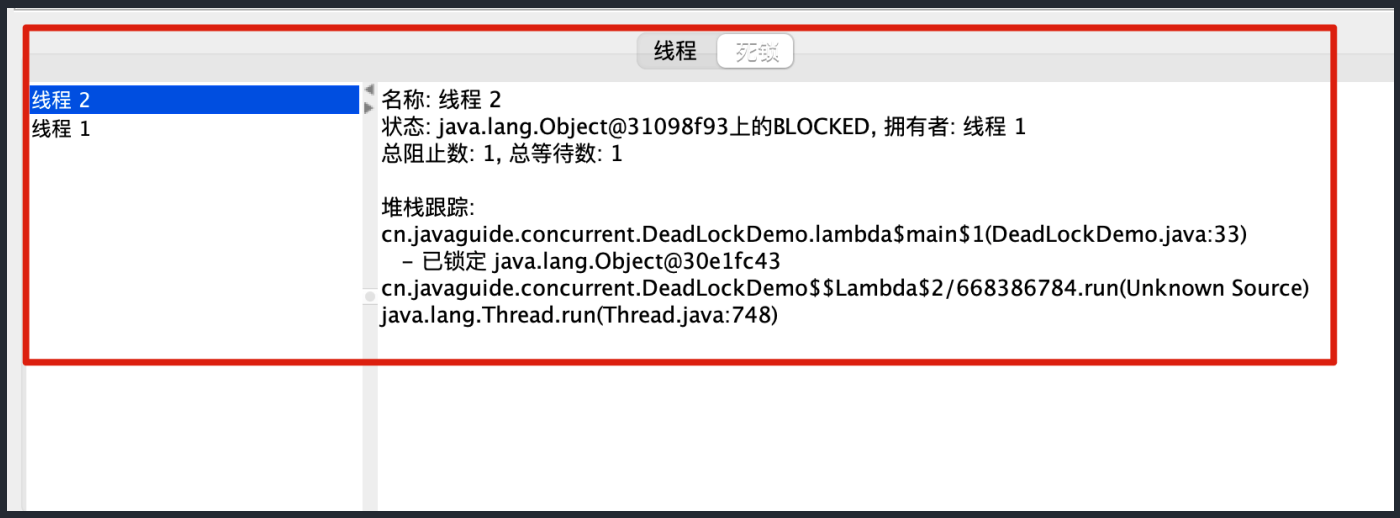


对于 MAC 用户来说，可以通过 `/usr/libexec/java_home -V` 查看 JDK 安装目录，找到后通过 `open . + 文件夹地址` 打开即可。例如，我本地的某个 JDK 的路径是：

```
open . /Users/guide/Library/Java/JavaVirtualMachines/corretto-1.8.0_252/Contents/Home
```

打开 jconsole 后，连接对应的程序，然后进入线程界面选择检测死锁即可！





详细介绍可以查看这篇文章的死锁部分内容：Java 并发常见面试题总结（上）。

什么是 Heap Dump 文件？如何生成 Heap Dump 文件？

Heap Dump（堆转储文件）是 Java 虚拟机（JVM）在某个特定时间点，对整个 Java **堆内存**的快照。它是一个二进制文件，包含了快照时刻堆中所有对象的信息，例如：

- **对象实例**：每个对象的数据。
- **类信息**：对象的类名、父类、静态字段等。
- **引用关系**：对象之间复杂的引用链，即谁持有了谁。
- **线程信息**：堆栈信息，特别是与 GC Roots 相关的线程栈。

简单来说，Heap Dump 就是 Java 进程在某一刻的“内存 X 光片”，是诊断内存问题的最核心、最权威的依据。

自动生成

在 JVM 启动参数中加入以下配置，这是生产环境排查 OOM 问题的首选方案。

```
# 当发生 OutOfMemoryError 时，自动生成 Heap Dump 文件
-XX:+HeapDumpOnOutOfMemoryError

# 指定 Heap Dump 文件的生成路径，例如：/home/app/dumps/
-XX:HeapDumpPath=<path-to-dump-dir>
```

手动生成

当应用出现内存疑似异常（如内存持续升高、GC 频繁）但未崩溃时，可以手动生成快照进行分析。

1. **jmap**：JDK 自带的命令行工具，专门用于生成堆快照。使用示例：`jmap -dump:format=b,file=heapdump.hprof <pid>`。在执行时会触发 STW，导致 Java 进程短暂停顿，对生产环境有一定影响。在高版本 JDK 中已不推荐直接使用。
2. **jcmd**：JDK 7 之后引入的多功能命令行工具，功能比 jmap 更强大一些，可用来替代 jmap，侵入性更小。使用示例：`jcmd <pid> GC.heap_dump /path/to/heapdump.hprof`。
3. **Arthas**：阿里巴巴开源的 Java 诊断神器，对应用无侵入，功能强大，可在不重启服务的情况下动态分析。使用示例：`heapdump /tmp/heapdump.hprof`。
4. **可视化工具**：如 JVisualVM、JProfiler、YourKit 等，都提供了图形化界面，点击按钮即可生成 Heap Dump 文件，并能直接进行分析，非常方便。

遇到 OutOfMemoryError 怎么排查解决?

我们可以通过 MAT、JVisualVM 等工具分析 Heap Dump 找到导致 `OutOfMemoryError` 的原因。

以 MAT 为例，其提供的泄漏嫌疑（Leak Suspects）报告是 MAT 最强大的功能之一。它会基于启发式算法自动分析整个堆，直接指出最可疑的内存泄漏点，并给出详细的报告，包括问题组件、累积点（Accumulation Point）和引用链的图示。

如果“泄漏嫌疑”报告不够明确，或者想要分析的是内存占用过高（而非泄漏）问题，可以切换到**支配树（Dominator Tree）**视图。这个视图将内存对象关系组织成一棵树，父节点“支配”子节点（即父节点被回收，子节点也必被回收）。

下面是一段模拟出现 `OutOfMemoryError` 的代码：

```java
import java.util.ArrayList;
import java.util.List;

public class SimpleLeak {

    // 静态集合，生命周期与应用程序一样长
    public static List<byte[]> staticList = new ArrayList<>();

    public void leakMethod() {
        // 每次调用都向静态集合中添加一个 1MB 的字节数组
        staticList.add(new byte[1024 * 1024]); // 1MB
    }

    public static void main(String[] args) throws InterruptedException {
        SimpleLeak leak = new SimpleLeak();
        System.out.println("Starting leak simulation...");

        // 循环添加对象，模拟内存泄漏过程
        for (int i = 0; i < 200; i++) {
            leak.leakMethod();
            System.out.println("Added " + (i + 1) + " MB to the list.");
            Thread.sleep(200); // 稍微延时，方便观察
        }
```

```
            System.out.println("Leak simulation finished. Keeping process alive for Heap Dump.");
            // 保持进程存活，以便我们有时间生成 Heap Dump
            Thread.sleep(Long.MAX_VALUE);
        }
    }
```

为了更快让程序出现 `OutOfMemoryError` 问题，我们可以故意设置一个较小的堆 `-Xmx256m` 。

IDEA 设置 VM 参数的方式如下图所示：

具体设置的 VM 参数是： `-Xmx128m -XX:+HeapDumpOnOutOfMemoryError -XX:HeapDumpPath=simple_leak.hprof` ，其中：

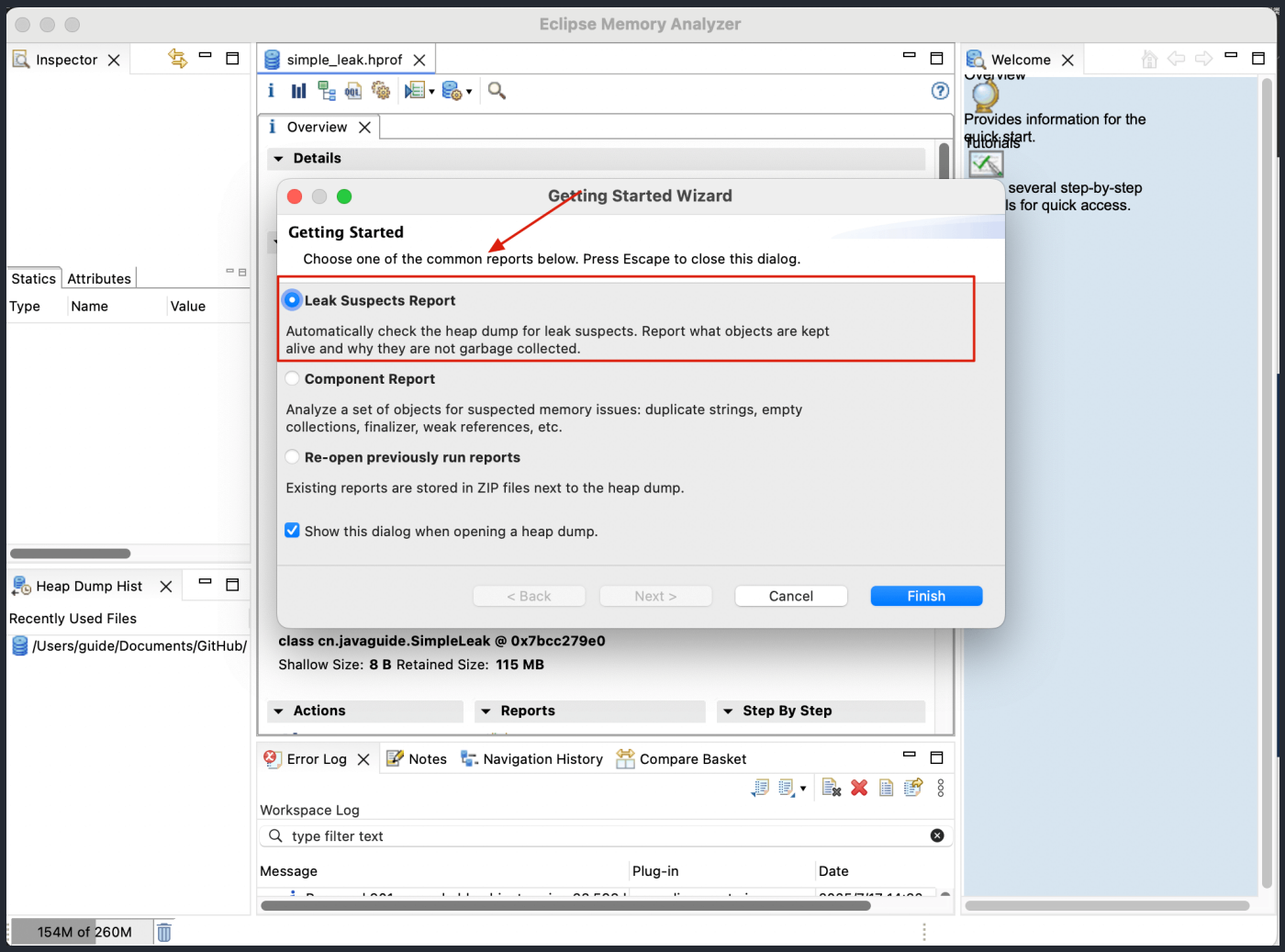
- `-Xmx128m` ：设置 JVM 最大堆内存为 128MB。
- `-XX:+HeapDumpOnOutOfMemoryError` ：当 JVM 发生 `OutOfMemoryError` 时，自动生成堆转储文件（ `.hprof` ）。

- `-XX:HeapDumpPath=simple_leak.hprof`：指定 OOM 时生成的堆转储文件路径及文件名（这里是 `simple_leak.hprof`）。

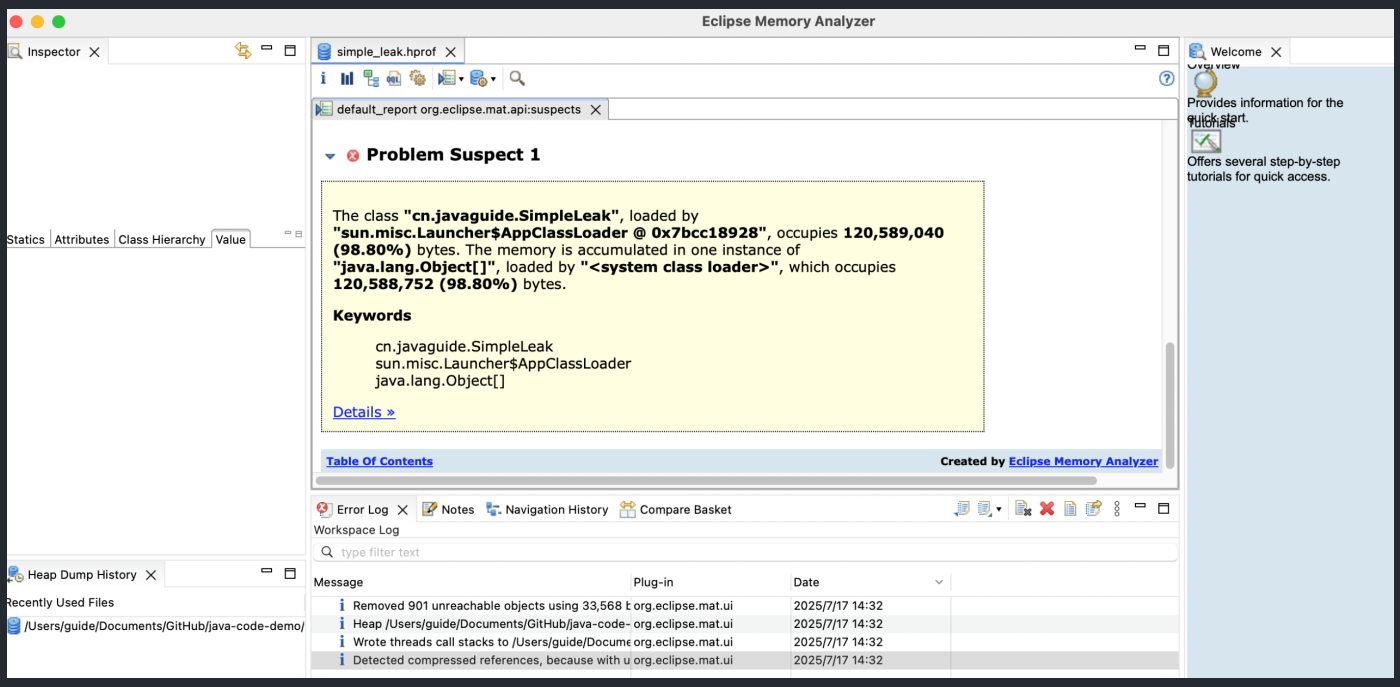
运行程序之后，会出现 `OutOfMemoryError` 并自动生成了 Heap Dump 文件。

```
Starting leak simulation...
Added 1 MB to the list.
Added 2 MB to the list.
Added 3 MB to the list.
......
Added 113 MB to the list.
Added 114 MB to the list.
Added 115 MB to the list.
java.lang.OutOfMemoryError: Java heap space
Dumping heap to simple_leak.hprof ...
Heap dump file created [124217346 bytes in 0.121 secs]
```

我们将 `.hprof` 文件导入 MAT 后，它会首先进行解析和索引。完成后，可以查看它的 **“泄漏嫌疑报告” (Leak Suspects Report)**。



下图中的 Problem Suspect 1 就是可能出现内存泄露的问题分析：



- `cn.javaguide.SimpleLeak` 类由 `sun.misc.Launcher$AppClassLoader` 加载，占用 **120,589,040 字节（约 115MB，占堆 98.80%）**，是内存占用的核心。

- 内存主要被 `java.lang.Object[]` **数组** 占用（120,588,752 字节），说明 `SimpleLeak` 中可能存在大量 `Object` 数组未释放，触发内存泄漏。

Problem Suspect 1 的可以看到有一个 **Details**，点进去即可看到内存泄漏的关键路径和对象占比：

Shortest Paths To the Accumulation Point

Class Name	Shallow Heap	Retained Heap
java.lang.Object[163] @ 0x7bd19d430	672	120,588,752
elementData java.util.ArrayList @ 0x7bcc278f8	24	120,588,776
staticList class cn.javaguide.SimpleLeak @ 0x7bcc279e0	8	120,589,040
appClass class sun.launcher.LauncherHelper @ 0x7bcc27920 System Class	80	912
<class> cn.javaguide.SimpleLeak @ 0x7bcc278e8 »	16	16
[3] java.lang.Object[10] @ 0x7bcd4acf8 »	56	608
Total: 3 entries		

Accumulated Objects in Dominator Tree

Class Name	Shallow Heap	Retained Heap	Percentage
class cn.javaguide.SimpleLeak @ 0x7bcc279e0	8	120,589,040	98.80%
java.util.ArrayList @ 0x7bcc278f8	24	120,588,776	98.80%
java.lang.Object[163] @ 0x7bd19d430	672	120,588,752	98.80%
byte[1048576] @ 0x7b8001730	1,048,592	1,048,592	0.86%
byte[1048576] @ 0x7b8101740	1,048,592	1,048,592	0.86%
byte[1048576] @ 0x7b8201750	1,048,592	1,048,592	0.86%
byte[1048576] @ 0x7b8322e08	1,048,592	1,048,592	0.86%
byte[1048576] @ 0x7b84345e8	1,048,592	1,048,592	0.86%
byte[1048576] @ 0x7b8544c80	1,048,592	1,048,592	0.86%
byte[1048576] @ 0x7b864a918	1,048,592	1,048,592	0.86%
byte[1048576] @ 0x7b874c900	1,048,592	1,048,592	0.86%
byte[1048576] @ 0x7b884e900	1,048,592	1,048,592	0.86%
byte[1048576] @ 0x7b89528c8	1,048,592	1,048,592	0.86%
byte[1048576] @ 0x7b8a58860	1,048,592	1,048,592	0.86%
byte[1048576] @ 0x7b8b833b0	1,048,592	1,048,592	0.86%
byte[1048576] @ 0x7b8cab580	1,048,592	1,048,592	0.86%
byte[1048576] @ 0x7b8daf548	1,048,592	1,048,592	0.86%
byte[1048576] @ 0x7b8edaec8	1,048,592	1,048,592	0.86%

可以看到：`SimpleLeak` 中的**静态集合** `staticList` 是内存泄漏的 “根源”，因为静态变量生命周期与类一致，若持续向其中添加对象且不清理，会导致对象无法被 GC 回收。

遇到过 GC 问题吗？怎么分析和解决的？

美团技术团队的 Java 中 9 种常见的 CMS GC 问题分析与解决这篇文章共 2w+ 字，详细介绍了 GC 基础，总结了 CMS GC 的一些常见问题分析与解决办法。